

SEARCH RELEVANCE · RETRIEVAL · LTR · ANN · RAG

搜索引擎算法岗 面试宝典

100 道核心真题与系统解法

从倒排索引、BM25、LambdaMART 到 HNSW、Hybrid Search、RAG
与系统设计

题解型 · 追问型 · 工业实践型

面向算法工程师 / Search Relevance / Retrieval / RAG Engineer

2026 Edition

使用说明与原创声明

本书采用“问题驱动、递进追问、短答案 + 深答案”的训练组织方式，借鉴技术面试题解书常见的学习节奏，但正文、题解、图表与版式均为原创整理，不复制《剑指 Offer》或其他出版物的原文、例题解析或页面设计。

“真题 / 高频题型”来自公开面试经验中长期稳定出现的知识点，以及当前 Search / Relevance / Retrieval 岗位的核心能力模型。公开面经无法像公司官方题库一样逐题认证，因此本书更适合被理解为：**高置信度面试题型 + 工业级知识框架**。

建议用法：第一遍只看“30 秒回答”；第二遍看“深度拆解”；第三遍只看“追问链”并脱稿作答；最后用第十章做系统设计模拟。

序言：算法岗真正考什么

搜索面试不是 100 个孤立知识点，而是一条完整因果链：

Query → Understanding → Recall → Rank → ReRank → Experiment

以及两条支撑链：**Indexing / Serving** 与 **Logs / Labels / Training**。

高级候选人与普通候选人的差别，通常不在于能否背出 BM25 公式，而在于：

1. 能否从公式解释行为，而不是只背符号；
2. 能否把离线指标、线上用户行为和偏差联系起来；
3. 能否做数量级估算与延迟/内存/召回权衡；
4. 能否看到 cascade 中错误如何传播；
5. 能否把模型、数据、系统和 A/B 组成持续优化闭环。

100 题能力地图

题号	模块	核心能力
1-10	全局架构	搜索 Pipeline、多阶段排序、Search vs RAG
11-20	倒排 / Lucene	Postings、压缩、FST、Segment、Merge
21-30	BM25	TF-IDF、BM25 推导、参数、字段与失败模式
31-40	Query	分词、纠错、意图、扩展、LLM Rewrite、Suggest
41-52	LTR	Point/Pair/List、RankNet、LambdaRank、LambdaMART
53-62	评估	NDCG、MRR、MAP、Click Bias、IPS、A/B
63-74	Dense / ANN	Dual Encoder、Hard Negative、IVF、PQ、HNSW
75-84	Hybrid / RAG	RRF、SPLADE、Cross-Encoder、ColBERT、Agentic Search
85-94	Infra	Shard、NRT、CDC、Cache、Tail Latency
95-100	系统设计	Web / 电商 / Semantic / Hybrid / 0→1

推荐学习顺序

如果只剩 3 天：Q1 → Q11 → Q24 → Q31 → Q41 → Q46 → Q47 → Q56 → Q60 → Q65 → Q72 → Q73 → Q75 → Q77 → Q79 → Q95 → Q99 → Q100。

如果有 2 周：按章节学习，每天 6-10 题；每道题先口述 30 秒版，再写白板版，最后只看追问链模拟压力面。

Contents

序言：算法岗真正考什么	3
1 第一章搜索引擎全局架构	5
1.1 Q01 ·完整讲一下搜索引擎的端到端 Pipeline	6
1.2 Q02 ·搜索系统与推荐系统的本质区别是什么？	7
1.3 Q03 ·为什么搜索一定要有候选召回阶段？	8
1.4 Q04 ·召回、粗排、精排、重排分别优化什么？	9
1.5 Q05 ·为什么不能用数据库 LIKE 代替搜索引擎？	10
1.6 Q06 ·一次搜索请求从键盘到 SERP 发生了什么？	11
1.7 Q07 ·为什么现代搜索几乎都是 Multi-stage Ranking？	12
1.8 Q08 ·搜索系统的目标函数为什么是多目标的？	13
1.9 Q09 ·为什么搜索结果不能直接按 CTR 排？	14
1.10Q10 ·传统 Search 与 RAG Retrieval 的目标有什么不同？	15
2 第二章倒排索引与 Lucene 内核	16
2.1 Q11 ·什么是倒排索引？	17
2.2 Q12 ·为什么叫“倒排”？正排索引还有什么用？	18
2.3 Q13 ·Posting List 中一般存哪些信息？	19
2.4 Q14 ·两个有序 Posting List 的 AND 查询怎么做？	20
2.5 Q15 ·Posting List 很长时，WAND / Block-Max WAND 在做什么？	21
2.6 Q16 ·为什么 DocID 常用 gap/delta 编码？	22
2.7 Q17 ·Term Dictionary 为什么常用 FST，而不只是 HashMap？	23
2.8 Q18 ·Trie 为什么适合做 Search Autocomplete？	24
2.9 Q19 ·Lucene Segment 为什么设计成 immutable？	25
2.10Q20 ·Segment Merge 为什么既重要又危险？	26
3 第三章 TF-IDF、BM25 与词法检索	27
3.1 Q21 ·TF-IDF 的核心直觉是什么？	28
3.2 Q22 ·为什么 IDF 能衡量一个词的“辨识度”？	29
3.3 Q23 ·TF-IDF 的主要问题是什么？	30
3.4 Q24 ·写出 BM25，并解释每一项的意义	31
3.5 Q25 ·BM25 相比 TF-IDF 到底改进了什么？	32
3.6 Q26 ·BM25 中 k1 控制什么？如何调？	33
3.7 Q27 ·BM25 中 b 控制什么？如何理解 b=0 和 b=1？	34
3.8 Q28 ·Title 和 Body 应该如何联合打分？什么是 BM25F 思想？	35
3.9 Q29 ·BM25 会在哪些场景失败？	36
3.10Q30 ·为什么 BM25 在 2026 年仍然非常强？	37

4 第四章 Query Understanding 与 Query Rewrite	38
4.1 Q31 ·Query Understanding 通常包括哪些任务?	39
4.2 Q32 ·中文搜索分词为什么比英文更难?	40
4.3 Q33 ·搜索分词是不是越细越好?	41
4.4 Q34 ·拼写纠错如何设计 Candidate Generation 与 Ranking?	42
4.5 Q35 ·“苹果”这样的 Query 为什么难? 如何做意图消歧?	43
4.6 Q36 ·什么是 Query Expansion? 为什么会同时提升 Recall 和伤害 Precision?	44
4.7 Q37 ·Synonym、Query Rewrite、Query Expansion 有什么区别?	45
4.8 Q38 ·LLM 如何用于 Query Rewrite?	46
4.9 Q39 ·LLM Query Rewrite 最大的风险是什么?	47
4.10Q40 ·如何设计 Search Autocomplete / Query Suggest?	48
5 第五章 Learning to Rank: 从 RankNet 到 LambdaMART	49
5.1 Q41 ·为什么需要 Learning to Rank? BM25 不够吗?	50
5.2 Q42 ·Pointwise Ranking 是什么? 优缺点?	51
5.3 Q43 ·Pairwise Ranking 是什么?	52
5.4 Q44 ·Listwise Ranking 是什么? 为什么更贴近 NDCG?	53
5.5 Q45 ·RankNet 的核心公式和直觉是什么?	54
5.6 Q46 ·LambdaRank 为什么出现? Lambda 到底是什么?	55
5.7 Q47 ·LambdaMART 是什么? 为什么经典?	56
5.8 Q48 ·深度学习时代, 为什么 LambdaMART 仍然常见?	57
5.9 Q49 ·搜索 Ranker 常见特征有哪些? 如何分类?	58
5.10Q50 ·什么是 Query-independent Feature? 为什么有价值?	59
5.11Q51 ·搜索排序中的 Feature Leakage 是什么?	60
5.12Q52 ·Ranker 为什么有时需要 Calibration?	61
6 第六章搜索指标、点击偏差与实验	62
6.1 Q53 ·Precision@K 与 Recall@K 分别衡量什么?	63
6.2 Q54 ·MRR 是什么? 适合什么场景?	64
6.3 Q55 ·MAP 是什么? 与 MRR 有何不同?	65
6.4 Q56 ·NDCG 是什么? 为什么是搜索面试必考?	66
6.5 Q57 ·为什么 NDCG 比 Accuracy 更适合 Search?	67
6.6 Q58 ·Recall@1000 提升但 NDCG@10 下降, 怎么解释?	68
6.7 Q59 ·为什么 Click 不能直接当 Relevance Label?	69
6.8 Q60 ·如何处理 Position Bias? 什么是 IPS?	70
6.9 Q61 ·Offline 指标涨了, Online CTR/满意度为什么可能下降?	71
6.10Q62 ·搜索 A/B Test 应如何设计?	72
7 第七章 Dense Retrieval 与 ANN	73
7.1 Q63 ·Dense Retrieval 与 BM25 的本质区别是什么?	74
7.2 Q64 ·为什么 Dual Encoder 适合召回?	75
7.3 Q65 ·双塔检索模型通常怎么训练?	76

7.4	Q66 ·为什么 Hard Negative 对 Dense Retrieval 至关重要?	77
7.5	Q67 ·什么是 In-batch Negative? 有什么坑?	78
7.6	Q68 ·Cosine、Inner Product、L2 在归一化向量下有什么关系?	79
7.7	Q69 ·为什么十亿向量不能直接暴力扫描? 如何算量?	80
7.8	Q70 ·IVF 的原理是什么? nlist 与 nprobe 如何影响效果?	81
7.9	Q71 ·Product Quantization (PQ) 是什么?	82
7.10	Q72 ·HNSW 的原理是什么? 为什么分层?	83
7.11	Q73 ·HNSW 的 M、efConstruction、efSearch 分别控制什么?	84
7.12	Q74 ·HNSW 与 IVF-PQ 如何选?	85
8	第八章 Hybrid Search、Neural ReRank 与 RAG	86
8.1	Q75 ·为什么 Hybrid Search 往往比纯 BM25 或纯 Dense 更稳?	87
8.2	Q76 ·BM25 Score 与 Dense Cosine 能直接相加吗?	88
8.3	Q77 ·什么是 Reciprocal Rank Fusion (RRF)?	89
8.4	Q78 ·SPLADE 这类 Learned Sparse Retrieval 在做什么?	90
8.5	Q79 ·为什么 Cross-Encoder 适合 Rerank, 而不适合全库 Recall?	91
8.6	Q80 ·Dual Encoder 与 Cross-Encoder 的经典 Trade-off 是什么?	92
8.7	Q81 ·ColBERT 的 Late Interaction 为什么重要?	93
8.8	Q82 ·RAG 的 Chunk Size 应该怎么选?	94
8.9	Q83 ·Retrieval Recall 很高, 为什么 RAG 仍会答错?	95
8.10	Q84 ·什么是 Agentic / Iterative Search?	96
9	第九章分布式搜索与工程系统	97
9.1	Q85 ·为什么 Search Index 要做 Sharding?	98
9.2	Q86 ·分布式 Search Query 的 Scatter-Gather 怎么工作?	99
9.3	Q87 ·为什么每个 Shard 只返回 Local TopK 可能有问题?	100
9.4	Q88 ·Primary Shard 与 Replica 的区别是什么?	101
9.5	Q89 ·Shard 越多是不是查询越快? 什么是 Over-sharding?	102
9.6	Q90 ·什么是 Near Real-Time (NRT) Search?	103
9.7	Q91 ·Refresh Interval 为什么存在 Freshness-Throughput Trade-off?	104
9.8	Q92 ·搜索索引如何与 MySQL/业务数据库保持同步?	105
9.9	Q93 ·搜索系统有哪些 Cache? 为什么 Query Result Cache 不一定最有效?	106
9.10	Q94 ·搜索延迟从 50ms 突然变成 2s, 怎么系统排查?	107
10	第十章综合系统设计与 0→1 方法论	108
10.1	Q95 ·系统设计: 从 0 设计一个 Google-like Web Search Engine	109
10.2	Q96 ·系统设计: 淘宝 / Amazon 商品搜索	110
10.3	Q97 ·系统设计: 亿级 Query Autocomplete	111
10.4	Q98 ·系统设计: 10 亿文档 Semantic Search	112
10.5	Q99 ·系统设计: 现代 Hybrid Search Engine	113
10.6	Q100 ·终极题: 如果让你从 0 到 1 提升一个搜索引擎, 你会怎么做?	114

11附录 A：核心公式速查	115
12附录 B：系统设计答题模板	116
13附录 C：14 天冲刺计划	117
14附录 D：参考资料	118
结语：把 100 题压缩成 10 个问题	119

CHAPTER 1

第一章搜索引擎全局架构

从端到端心智模型开始，建立“离线索引 + 在线级联 + 日志闭环”的整体框架。

题号	题目	难度
Q01	完整讲一下搜索引擎的端到端 Pipeline	3/5
Q02	搜索系统与推荐系统的本质区别是什么？	2/5
Q03	为什么搜索一定要有候选召回阶段？	3/5
Q04	召回、粗排、精排、重排分别优化什么？	3/5
Q05	为什么不能用数据库 LIKE 代替搜索引擎？	2/5
Q06	一次搜索请求从键盘到 SERP 发生了什么？	3/5
Q07	为什么现代搜索几乎都是 Multi-stage Ranking？	3/5
Q08	搜索系统的目标函数为什么是多目标的？	4/5
Q09	为什么搜索结果不能直接按 CTR 排？	4/5
Q10	传统 Search 与 RAG Retrieval 的目标有什么不同？	4/5

Q01 · 完整讲一下搜索引擎的端到端 Pipeline

难度 3/5 频率 S 适用岗位通用 / 搜索架构

30 秒回答

搜索不是“输入关键词后查数据库”，而是一条离线索引链和一条在线查询链的汇合：离线侧把文档解析、标准化并构建倒排/向量索引；在线侧做 Query Understanding、候选召回、粗排、精排和重排；日志再反哺训练与实验，形成闭环。

深度拆解

- 离线链路：采集/接入 → 解析与清洗 → 分词与字段化 → 倒排索引、向量索引、特征库 → 分片与发布。
- 在线链路：Query 标准化 → 意图/实体/纠错/改写 → 多路召回 → 候选合并 → PreRank → Rank → ReRank → 摘要、高亮与结果返回。
- 训练闭环：曝光/点击/停留/转化日志 → 去偏与标注 → Retriever/Ranker 训练 → 离线评估 → A/B → 新日志。
- 系统设计时必须同时回答五件事：相关性、召回率、延迟、索引新鲜度、成本。
- 高级面试官通常不满足于“画流程图”，会追问每一层为什么存在，以及如果删除某层会发生什么。

- 为什么不直接让一个大模型对全库排序？
- 倒排索引和向量索引分别在什么阶段更新？
- 召回层的 TopK 与精排层的 TopK 如何设？

最常见失分是只讲在线搜索，不讲索引构建、日志闭环与实验系统；或者把 Elasticsearch 当成搜索算法本身。

搜索 = Offline Indexing + Online Cascade + Feedback Loop。

参考：R1, R14

Q02 · 搜索系统与推荐系统的本质区别是什么？

难度 2/5 频率 A 适用岗位搜索 / 推荐交叉

30 秒回答

两者都可以做候选生成和排序，但搜索多了一个极强的显式意图变量 Query。搜索首先要满足 query-document relevance，再叠加个性化、质量、新鲜度和业务目标；推荐则更多从用户长期/短期兴趣中推断“现在可能想看什么”。

深度拆解

- 搜索建模可写成 $P(d | q, u, c)$ ，推荐更接近 $P(d | u, c)$ ； q 往往是搜索中信息量最高的变量。
- 搜索更怕“答非所问”：即使一个商品很热门，如果与 query 不相关也不能排前。
- 推荐更强调探索、兴趣扩展与长期留存；搜索更强调满足当前任务。
- 电商搜索会逐渐向推荐靠拢：同一个“鞋子”query，不同用户的品牌、价格带、尺码偏好会不同。
- 工程上二者都会使用多路召回、GBDT/DNN 排序、实时特征和 A/B，但标签构造与评价函数不同。

- 电商搜索可以多大程度个性化？
- Query 很明确时，用户画像应该占多大权重？
- Search 与 Feed 为什么常共用部分特征平台？

把搜索说成“有 Query 的推荐”不够完整；必须强调 relevance 是硬约束或首要目标。

推荐猜需求，搜索兑现需求。

Q03 · 为什么搜索一定要有候选召回阶段？

难度 3/5 频率 S 适用岗位搜索 / RAG

30 秒回答

因为全库精排的计算量与延迟不可接受。召回阶段用便宜的检索函数把十亿级文档缩小到千级/万级候选，目标是高 Recall；后续再用昂贵模型提高排序精度。

深度拆解

- 若文档数为 N ，Cross-Encoder 对全库逐一计算的近似成本 $O(N \cdot C_{model})$ 。
- 倒排索引利用 $\text{term} \rightarrow \text{postings}$ 只访问与 query term 相关的文档；ANN 只探索向量空间的一小部分。
- 召回阶段的核心错误是 **false negative**：相关文档一旦没进入候选，后面的 Ranker 再强也救不回来。
- 因此常见优化顺序是先看 $\text{Recall}@1000/\text{Recall}@5000$ ，再看 $\text{NDCG}@10$ 。
- 多路召回是工业常态：BM25、Dense、规则/实体、热门、新鲜内容等并行，再合并去重。

- 召回 TopK 越大越好吗？
- $\text{Recall}@1000$ 很高但线上效果差，可能为什么？
- 如何衡量不同召回通道的增量价值？

只说“为了快”太浅；应明确说明 cascade 的经济性：前层高召回、低成本，后层高精度、高成本。

先“别漏”，再“排对”。

Q04 · 召回、粗排、精排、重排分别优化什么？

难度 3/5 频率 S 适用岗位 排序架构

30 秒回答

四层不是模型大小的简单递增，而是约束不同：召回保覆盖，粗排做低成本淘汰，精排最大化相关性与业务收益，重排处理列表级约束。

深度拆解

- 召回：重点看 Recall@K、Coverage、通道增量与延迟。
- 粗排：重点是单位计算成本的排序能力，常用轻量 GBDT/DNN 或蒸馏模型。
- 精排：重点优化 NDCG、CTR/CVR、long-click 等目标，可使用 LambdaMART、深度排序模型、Cross-Encoder。
- 重排：处理 diversity、去重、freshness、类目配额、合规策略和多目标约束，属于 list-level optimization。
- 每层都要有独立诊断指标，否则“整体 NDCG 降了”无法定位是召回漏了还是排序排错。

- PreRank 的训练标签能否与 Rank 完全相同？
- 为什么 ReRank 不应该只看 pointwise score？
- 如何选择每层的候选数？

不要把 ReRank 等同于“再跑一个更大的模型”；它经常承担业务与列表约束。

Recall 负责入场，**Rank** 负责座次，**ReRank** 负责整桌体验。

Q05 · 为什么不能用数据库 LIKE 代替搜索引擎？

难度 2/5 频率 A 适用岗位搜索基础

30 秒回答

数据库索引主要解决结构化 lookup；全文搜索需要 tokenization、倒排索引、相关性评分、短语/位置检索、纠错与排序。LIKE

深度拆解

- B-tree 的优势是有序键上的点查、范围查；前导通配符会破坏可利用的有序前缀。
- 倒排索引把 term 映射到 posting list，使查询只扫描包含相关 term 的文档。
- 全文检索还需要 TF、DF、字段长度、positions、offsets 等统计与结构。
- 搜索结果不是“匹配/不匹配”二值结果，而是需要 TopK 排序。
- 数据库也能做全文索引，但当需求进入多字段相关性、ANN、LTR、分布式检索后，通常需要专门搜索系统。

- 前缀 LIKE 'abc
- 数据库全文索引和 Lucene 有什么差异？
- 什么时候直接 SQL 已经足够？

面试中不要绝对说“LIKE 一定全表扫”；是否走索引取决于模式、collation、数据库实现与查询计划。

数据库找“哪一行”，搜索引擎找“哪一行最值得先看”。

Q06 · 一次搜索请求从键盘到 SERP 发生了什么？

难度 3/5 频率 A 适用岗位搜索架构

30 秒回答

高质量回答应从入口网关一直讲到 shard fan-out、候选合并与摘要生成，并指出哪些步骤是同步关键路径、哪些可以缓存或异步。

深度拆解

- 入口层：鉴权、限流、AB bucket、query 日志、地域/设备上下文。
- Query Understanding: normalize、language、spelling、segmentation、intent、entity、rewrite。
- 检索层：向 lexical shards / vector shards 并发请求，得到局部 TopK。
- 协调层：dedup、fusion、feature fetch、PreRank/Rank/ReRank, 必要时调用 GPU model service。
- 展示层：snippet/highlight、结果模板、过滤敏感或失效文档。
- 可观测性：记录 stage latency、candidate counts、cache hit、模型版本和最终曝光。

- 哪一步最容易造成 p99 尾延迟？
- 特征服务超时如何降级？
- 某一个 shard 超时是返回部分结果还是整单失败？

只讲算法不讲超时、降级与可观测性，会在高级岗位系统设计中明显失分。

把“算法链路”和“关键路径工程”一起讲。

Q07 · 为什么现代搜索几乎都是 Multi-stage Ranking?

难度 3/5 频率 S 适用岗位排序架构

30 秒回答

因为计算预算有限且候选数量逐层减少。不同阶段使用不同模型，让单位延迟获得最大的质量收益，这是经典 cascade architecture。

深度拆解

- 召回对百万/十亿规模，必须使用亚线性或稀疏访问；精排只面对几百候选。
- 模型越昂贵，适用的候选集就越小：BM25/ANN → GBDT/bi-encoder → cross-encoder/LLM。
- 如果前层过于激进，会产生不可恢复的 recall loss；如果过于宽松，会把计算压力推给下游。
- 阶段间通常要做蒸馏或目标对齐，避免粗排把精排偏好的候选误删。
- 工业优化常用 “quality-latency frontier”，而不是单点最大 NDCG。

- 如何做 Ranker 到 PreRank 的蒸馏？
- 候选数变化为什么会影响模型分布？
- 什么时候可以省掉粗排？

不要机械说“四层固定”；真实系统可能是 2 层、3 层或更多，取决于 QPS、候选量和模型成本。

多阶段排序是在算力预算内做最优决策。

Q08 · 搜索系统的目标函数为什么是多目标的？

难度 4/5 频率 A 适用岗位排序 / 业务

30 秒回答

相关性是基础，但真实系统还要兼顾质量、新鲜度、可信度、多样性、点击/转化与延迟。常见做法是多任务预测、线性/非线性融合或约束优化。

深度拆解

- 可抽象为 $U = \alpha R + \beta E + \gamma Q + \delta F - \lambda C$ ，其中 C 可以代表延迟/成本或风险。
- 相关性通常不能被业务目标完全覆盖，否则会出现“高转化但不相关”的劣化。
- 不同 query intent 权重应不同：导航型更重准确命中，新闻型更重 freshness，电商型还看价格/库存/转化。
- 多目标训练可用 shared-bottom/MTL，但最终 list-level policy 常仍需显式规则或约束。
- 长期指标与短期指标可能冲突，例如提高标题党 CTR 会损害 long-click 与留存。

- CTR 与 relevance 如何解耦？
- 如何给 freshness 设动态权重？
- 多目标融合为什么需要 calibration？

“把所有指标加权求和”只是起点；真正难点是权重随意图变化、尺度校准与不可违反的硬约束。

先守住 **relevance**，再优化 **utility**。

Q09 · 为什么搜索结果不能直接按 CTR 排？

难度 4/5 频率 S 适用岗位 LTR / 用户行为

30 秒回答

CTR 不是纯相关性标签。它受到曝光位置、样式、品牌、历史排序与用户选择偏差影响。直接按 CTR 排会形成反馈回路：排得高 → 点击多 → 继续排高。

深度拆解

- 点击可分解为 $P(\text{click}) = P(\text{examine}) \cdot P(\text{click} \mid \text{examine}, \text{relevance}, \dots)$ 。
- Position bias: 顶部结果更容易被看到；trust bias: 用户更信任前几位；presentation bias: 卡片样式会改变点击。
- Popularity bias 会持续强化头部文档，压制新内容与长尾。
- 训练时可采用随机化实验、propensity estimation、IPS、pairwise click models 或人工 judgment。
- 线上常结合 click、dwell time、reformulation、conversion 等多信号判断满意度。

- Long click 为什么比 click 更接近满意度？
- 如何估计 propensity？
- 随机打乱排序会不会伤害用户？

“CTR 高 = 相关”是搜索面试的典型陷阱。

点击是有偏观察，不是相关性真值。

参考: R4

Q10 · 传统 Search 与 RAG Retrieval 的目标有什么不同？

难度 4/5 频率 A 适用岗位 RAG / 搜索

30 秒回答

传统搜索优化用户看到的结果列表；RAG Retrieval 的结果会继续被生成模型消费，因此评价要延伸到 evidence coverage、context precision 与最终 answer correctness。

深度拆解

- 传统搜索更关心 NDCG/MRR/CTR；RAG 还关心 answer faithfulness、evidence recall、context utilization。
- RAG 中“相关文档”不等于“足够回答问题的证据”，需要考虑多跳、互补证据与 chunk 粒度。
- 过多相似 chunk 会占满 context window，造成冗余和 lost-in-context。
- Reranker 的最优目标可能从“文档相关性”变成“对回答是否提供边际证据”。
- 因此 RAG 的检索评估最好做 end-to-end：Retriever → Reranker → Context Builder → Generator。

- 如何定义 context recall？
- Retriever 变好但 Answer Accuracy 不变可能为什么？
- RAG 是否应该用 query expansion？

不要把 RAG 简化成“向量搜索”；BM25、hybrid、reranking 和过滤同样重要。

Search 排文档；**RAG** 排“对回答有用的证据”。

第二章倒排索引与 Lucene 内核

理解搜索为什么能在海量文本上低延迟工作，以及性能优化真正发生在哪里。

题号	题目	难度
Q11	什么是倒排索引？	2/5
Q12	为什么叫“倒排”？正排索引还有什么用？	1/5
Q13	Posting List 中一般存哪些信息？	3/5
Q14	两个有序 Posting List 的 AND 查询怎么做？	2/5
Q15	Posting List 很长时，WAND / Block-Max WAND 在做什么？	5/5
Q16	为什么 DocID 常用 gap/delta 编码？	3/5
Q17	Term Dictionary 为什么常用 FST，而不只是 HashMap？	4/5
Q18	Trie 为什么适合做 Search Autocomplete？	2/5
Q19	Lucene Segment 为什么设计成 immutable？	4/5
Q20	Segment Merge 为什么既重要又危险？	4/5

Q11 · 什么是倒排索引?

难度 2/5 频率 S 适用岗位 IR 基础

30 秒回答

倒排索引把“文档包含哪些词”的正向关系反转成“每个词出现在哪些文档中”。核心结构是 Term Dictionary + Posting Lists。

深度拆解

- 正排: $doc \rightarrow terms$; 倒排: $term \rightarrow postings$ 。
- posting 最少包含 docID, 常进一步包含 term frequency、positions、offsets、payload。
- 查询 'A AND B' 变成两个有序 posting list 的交集, 而不是扫描所有文档。
- 索引构建阶段要做 analyzer: normalization、tokenization、stemming/lemmatization、stopword 等。
- 倒排索引是 BM25、phrase query、highlight 和多字段搜索的基础。

- 为什么 posting list 要有序?
- positions 会增加多少空间?
- 中文分词变化后为什么必须重建索引?

只回答“词到文档 ID 的映射”拿不到高分; 应提 Term Dictionary、posting 元数据与查询执行。

倒排索引 = Dictionary 定位词 + Posting 定位文档。

参考: R1, R14

Q12 · 为什么叫“倒排”？正排索引还有什么用？

难度 1/5 频率 B 适用岗位 IR 基础

30 秒回答

“倒排”是相对文档到词项的正排关系而言。正排仍用于文档展示、字段取回、摘要生成和部分特征读取；倒排主要服务高效检索。

深度拆解

- Forward index 适合从 docID 读取文档字段。
- Inverted index 适合从 term 找候选文档。
- 真实搜索系统通常同时保存 source/document store、doc values/columnar features 与 inverted index。
- Lucene 中 stored fields、doc values 与 postings 各承担不同访问模式。

- 为什么排序特征适合列式存储？
- Stored field 和 postings 为什么不合成一个结构？

不要误以为倒排索引可以高效恢复完整原文。

正排负责“取内容”，倒排负责“找候选”。

Q13 · Posting List 中一般存哪些信息？

难度 3/5 频率 A 适用岗位 Lucene / IR

30 秒回答

posting 的字段设计决定能支持哪些查询和打分。最基本是 docID；要做 BM25 需要词频，要做 phrase/proximity 需要 positions，要做 highlight 需要 offsets。

深度拆解

- docID：文档标识，通常排序后做差分压缩。
- tf/freq：term 在该文档中的出现次数。
- positions：每次出现的 token position，支持 phrase 和 near query。
- offsets：字符起止位置，用于高亮和精确片段。
- payload：可附加自定义信息，但会增加索引空间与查询成本。
- Index options 常是功能与空间的 trade-off：不是每个字段都需要 positions/offsets。

- 只存 docID 能不能算 BM25？
- Phrase Query 最少需要什么信息？
- 为什么 postings 常按 block 存储？

面试中要把“支持的查询能力”和“存储成本”联系起来。

你想在线算什么，就要离线存什么。

Q14 · 两个有序 Posting List 的 AND 查询怎么做？

难度 2/5 频率 S 适用岗位算法手撕

30 秒回答

用双指针线性求交：较小 docID 的指针前进，相等时输出。复杂度 $O(|A| + |B|)$ ，在长短悬殊时可进一步用 skip/gallop。

深度拆解

- 基础算法：‘ $i=j=0$ ’，若 ‘ $A[i]=B[j]$ ’ 输出并同时前进；否则移动较小者。
- 当一条 posting 很短、另一条很长时，可以对长表二分或 exponential/gallop search。
- 多 term AND 可先按 posting length 从短到长求交，尽早缩小候选。
- 现代引擎还会使用 skip data、block metadata 与 SIMD 优化。

公式 / 复杂度

白板 / 手撕：

```
def intersect(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] == b[j]:
            out.append(a[i]); i += 1; j += 1
        elif a[i] < b[j]:
            i += 1
        else:
            j += 1
    return out
```

- OR 查询怎么做？
- 三条 posting list 的交集顺序如何选？
- 为什么 ‘AND’ 仍可能很慢？

仅写出双指针还不够，进阶回答要讨论数据分布与跳跃优化。

有序是倒排查询性能的根。

Q15 · Posting List 很长时，WAND / Block-Max WAND 在做什么？

难度 5/5 频率 A 适用岗位搜索性能

30 秒回答

它们利用评分上界做动态剪枝：如果某文档或某个 block 即使取到理论最大贡献也不可能超过当前 TopK 阈值，就跳过，不再做完整评分。

深度拆解

- TopK 检索不需要给所有匹配文档精确打分，只需要证明被跳过的候选不可能进入 TopK。
- WAND 为 term 维护 upper bound，通过 pivot 判断哪些 doc 可以被安全跳过。
- Block-Max WAND 将上界细化到 posting blocks，局部上界比全局上界更紧，剪枝更强。
- 当前 heap 中第 K 名分数越高，阈值越严格，后续越容易剪枝。
- 它解释了为什么高频词查询未必需要完整扫描所有匹配文档。

- 上界如果估计错误会怎样？
- 为什么 disjunction 更需要 WAND？
- TopK 越小剪枝通常越强还是越弱？

不要把 WAND 说成近似算法；在上界正确时，它可以做 safe pruning，仍返回精确 TopK。

TopK 搜索的关键是“证明无需评分”，不是“更快算评分”。

Q16 · 为什么 DocID 常用 gap/delta 编码？

难度 3/5 频率 A 适用岗位索引压缩

30 秒回答

有序 docID 的差值通常远小于原始 ID，因此更容易用变长整数或 bit packing 压缩。更小的索引不仅省磁盘，也能提高 cache locality 和解码吞吐。

深度拆解

- 例如 ‘100,105,108,120’ 可存成 ‘100,5,3,12’。
- 常见编码包括 Variable Byte、PForDelta、SIMD-BP128 等。
- 压缩不是单纯节省磁盘：posting 能驻留更多 page cache，查询往往更快。
- 块式编码通常以 128/256 个整数为单位，方便 SIMD 解码和跳跃。
- 压缩率取决于 term document frequency 与 docID assignment/locality。

- 为什么随机 DocID 分配会影响压缩率？
- Variable Byte 和 bit packing 的 trade-off？

“压缩一定更慢”是错误直觉；搜索中更少的内存带宽常常抵消解码成本。

索引压缩是在用少量 CPU 换大量内存带宽。

Q17 · Term Dictionary 为什么常用 FST，而不只是 HashMap?

难度 4/5 频率 A 适用岗位 Lucene

30 秒回答

搜索词典既要压缩大量共享前缀，又要支持有序遍历、prefix/automaton 查询。FST 将 trie-like 前缀共享与状态合并结合，在内存效率与功能上优于朴素 HashMap。

深度拆解

- HashMap 适合 exact lookup，但键本身存储冗余，prefix/range 遍历能力弱。
- Trie 共享前缀，但节点数可能很大；FST 可合并等价后缀状态，进一步压缩。
- 有序 term dictionary 对 range、wildcard、prefix enumeration 很重要。
- 搜索引擎通常还需要 term metadata 指向 postings 的文件位置。
- 面试中不需要手推 FST 最小化算法，但要说清“为什么不是 hash”。

- FST 与 Trie 区别？
- Wildcard query 为什么可能很贵？
- Term dictionary 能否完全放内存？

不要把 FST 仅仅理解为“更高级的 Trie”；其关键价值是有限状态机/转导器的共享与紧凑编码。

词典的目标不是只查得快，还要小、可遍历、可自动机匹配。

Q18 · Trie 为什么适合做 Search Autocomplete?

难度 2/5 频率 A 适用岗位 Query Suggest

30 秒回答

Autocomplete 是前缀检索。Trie 让查找前缀只与前缀长度有关,再从该节点向下枚举高质量 completion; 若每个节点缓存 TopN, 则能做到非常低延迟。

深度拆解

- 共享前缀能降低字符串存储重复。
- 节点可维护 top suggestions、频率、最近热度等聚合统计。
- 大规模系统常使用压缩 Trie/FST, 降低对象和指针开销。
- 个性化 autocomplete 常是“全局候选 + 用户历史候选 + 热点候选”融合。
- 需要额外处理 typo、语言混输、敏感词、趋势变化。

- 如何支持中间词补全而非纯前缀?
- 百万 QPS 下如何更新热度?
- TopN 缓存在每个节点会不会太占内存?

Trie 解决的是 candidate generation, 不等于完整 autocomplete 排序系统。

前缀结构负责找, **Ranker** 负责排。

Q19 · Lucene Segment 为什么设计成 immutable?

难度 4/5 频率 S 适用岗位 Lucene / 系统

30 秒回答

不可变 segment 让读路径非常简单：查询可并发 mmap/read，无需对 postings 做细粒度写锁；新增文档写入新 segment，更新通过删除旧版本 + 新增版本实现。

深度拆解

- 读写分离：已发布 segment 不再原地修改，减少并发控制复杂度。
- 文件不可变有利于 OS page cache、校验、复制和 crash recovery。
- 删除先记录在 deletion bitmap/tombstone 中，真正回收空间通常发生在 merge。
- 代价是 segment 数会增长，查询需要跨 segment 搜索，因此需要 merge policy。
- 更新成本本质上变成 write amplification，而不是随机修改 postings。

- NRT refresh 为什么会产生小 segment?
- Delete-heavy workload 会发生什么?
- 为什么 segment merge 不能无限并发?

不要说“Lucene 不能更新”；它能提供更新语义，只是底层通过新版本与删除标记实现。

Immutable segment 用写放大换并发读简单。

参考：R14

Q20 • Segment Merge 为什么既重要又危险？

难度 4/5 频率 S 适用岗位 Lucene / 性能

30 秒回答

Merge 把多个小 segment 合成大 segment，减少查询 fan-out 并回收删除空间；但它会重写大量索引文件，造成 CPU、磁盘 I/O 和写放大，因此常是 indexing 抖动的重要来源。

深度拆解

- 查询成本大致随 active segment 数增加：每个 segment 都有词典查找、posting scan 与 TopK 合并。
- Merge 会重新编码 postings、stored fields、doc values 等，并清除删除文档。
- 如果 merge 跟不上写入，会出现 segment backlog、磁盘占用上涨和查询 cache locality 下降。
- 过度激进 merge 会与前台 indexing/query 抢 I/O；过度保守则积累太多小 segment。
- 诊断 indexing latency 突增时要看 merge time、I/O utilization、segment count 和 throttling。

- 为什么 SSD 仍然会被 merge 打爆？
- Merge 策略如何权衡 size tiers？
- 索引只读场景是否需要 merge？

高级回答要把 merge 和 LSM 类系统中的 compaction 类比：本质都是后台重写换读性能。

Merge 是搜索索引的 compaction。

第三章 TF-IDF、BM25 与词法检索

从经典公式到字段建模与失败模式，理解 lexical relevance 的真实边界。

题号	题目	难度
Q21	TF-IDF 的核心直觉是什么？	2/5
Q22	为什么 IDF 能衡量一个词的“辨识度”？	2/5
Q23	TF-IDF 的主要问题是什么？	3/5
Q24	写出 BM25，并解释每一项的意义	4/5
Q25	BM25 相比 TF-IDF 到底改进了什么？	3/5
Q26	BM25 中 k_1 控制什么？如何调？	3/5
Q27	BM25 中 b 控制什么？如何理解 $b=0$ 和 $b=1$ ？	3/5
Q28	Title 和 Body 应该如何联合打分？什么是 BM25F 思想？	4/5
Q29	BM25 会在哪些场景失败？	3/5
Q30	为什么 BM25 在 2026 年仍然非常强？	3/5

Q21 · TF-IDF 的核心直觉是什么？

难度 2/5 频率 S 适用岗位 IR 基础

30 秒回答

一个词对文档的重要性由“在当前文档里出现得多”和“在整个语料里出现得少”共同决定，因此是局部频率 TF 与全局稀有度 IDF 的组合。

深度拆解

- $TF(t, d)$ 表示 term 在文档中的频率； $DF(t)$ 表示包含该 term 的文档数。
- 常见 $IDF(t) = \log \frac{N}{df_t}$ ：越稀有的 term 权重越高。
- 实际实现常对 TF 做 log/sublinear 变换，对向量做 cosine normalization。
- TF-IDF 不等于 BM25；BM25 进一步显式建模 TF saturation 与 length normalization。

- 为什么不能只用 TF？
- IDF 为负怎么办？
- TF-IDF 和 cosine similarity 什么关系？

公式记忆不是重点，关键是解释“局部重要 + 全局辨识度”。

TF 看文档内，IDF 看语料库。

Q22 · 为什么 IDF 能衡量一个词的“辨识度”？

难度 2/5 频率 A 适用岗位 IR 基础

30 秒回答

如果一个词出现在几乎所有文档中，它无法区分候选；如果只出现在很少文档中，一旦 query 命中它，就强烈缩小了候选空间。IDF 可理解为一种信息量近似。

深度拆解

- $df_t \uparrow \Rightarrow idf_t \downarrow$ 。
- 停止词如“the/的”通常 df 极高，因此贡献低。
- 实体名、型号、错误码等 rare terms 在 lexical retrieval 中非常有价值。
- IDF 依赖 collection，跨索引或跨 shard 的统计不一致会改变得分。

- 新词 $df=0$ 怎么处理？
- 分片局部 IDF 会有什么偏差？

稀有不等于高质量：乱码 token 也可能很稀有，所以 analyzer/质量控制仍重要。

IDF 衡量“这个词把候选缩小了多少”。

Q23 · TF-IDF 的主要问题是什么？

难度 3/5 频率 S 适用岗位 IR 基础

30 秒回答

朴素 TF-IDF 对词频增长过于线性，对长文档的天然优势处理不足，也没有把长度归一化与 term saturation 写成一个更合理的检索概率/相关性函数。BM25 正是在这些问题上做经典改进。

深度拆解

- 同一个 term 从 1 次增加到 2 次很有信息，但从 100 次增加到 101 次几乎不该继续等量增益。
- 长文档更容易偶然包含 query term，若不校正会被系统性高估。
- 朴素 TF-IDF 的不同实现差异较大：TF 变换、IDF smooth、cosine normalization 都会改变结果。
- 它仍是理解 sparse retrieval 的重要基线，但工业全文检索通常用 BM25 或变体。

- Cosine normalization 能否完全解决长度问题？
- BM25 为什么不需要再做 cosine normalization？

不要说“TF-IDF 没有长度归一化”绝对错误；经典向量空间模型可以用 cosine 做归一化。应说 BM25 对长度和 TF 的建模更直接、成熟。

BM25 的两把刀：TF 饱和 + 长度校正。

Q24 · 写出 BM25，并解释每一项的意义

难度 4/5 频率 S 适用岗位 IR / Lucene

30 秒回答

BM25 是搜索算法岗必背公式，但面试重点不是背符号，而是能从公式解释 rare term、TF saturation 与 length normalization 三个效应。Lucene 当前 BM25Similarity 默认 $k_1 = 1.2, b = 0.75$ 。

深度拆解

- 对 query 中每个 term 累加贡献。
- IDF 控制全局稀有度； $f(q_i, D)$ 是 term frequency。
- k_1 控制 TF 饱和速度：越大越接近“TF 继续有效”。
- b 控制长度归一化： $b = 0$ 不看长度， $b = 1$ 完全按相对字段长度校正。
- Lucene 的 IDF 形式为 $\log(1 + (N - df + 0.5)/(df + 0.5))$ ，避免某些情况下出现不直观的负值。

公式 / 复杂度

$$BM25(D, Q) = \sum_{q_i \in Q} IDF(q_i) \frac{f(q_i, D)(k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + b \frac{|D|}{avgdl}\right)}$$

- $k_1 \rightarrow 0$ 时会怎样？
- $b = 0$ 时长文与短文如何比较？
- 同一个词在 title 和 body 应如何处理？

不能把 k_1 和 b 的意义说反；这是非常常见的面试失误。

BM25 = rare-term bonus × saturated TF × length correction。

参考：R2

Q25 • BM25 相比 TF-IDF 到底改进了什么？

难度 3/5 频率 S 适用岗位 IR

30 秒回答

最核心的两个改进是：词频贡献非线性饱和，以及文档长度相对平均长度的显式归一化。它使“多出现几次”与“文档很长”对得分的影响更符合检索经验。

深度拆解

- TF saturation: term 出现次数从 1→2 比 20→21 更重要。
- Length normalization: 长文更容易出现词，因此同等 TF 下需要惩罚过长字段。
- BM25 的参数让这两个效应可调，而不是固定死。
- BM25 仍然是 bag-of-words: 它不会自动理解同义词和深层语义。

- BM25 是否考虑 term proximity?
- BM25 对 query term frequency 如何处理?
- BM25 的分数是否可跨 query 比较?

BM25 score 不是概率，通常也不适合直接跨 query 做阈值。

TF-IDF 是“乘法直觉”，BM25 是“饱和 + 长度”的工程化相关性。

Q26 · BM25 中 k_1 控制什么？如何调？

难度 3/5 频率 A 适用岗位 IR 调参

30 秒回答

k_1 控制 term frequency 的非线性饱和。较小 k_1 让一次或少数几次出现就接近饱和；较大 k_1 让额外词频继续带来更多增益。

深度拆解

- $k_1 = 0$ 时，TF 因子基本退化为“是否出现”，额外出现次数不再增益。
- 较大的 k_1 适合词频本身更有意义的字段，但也更容易奖励关键词堆砌。
- 参数最佳值依赖 tokenizer、字段和 domain，Lucene 默认值只是稳健起点。
- 调参应在 query-level validation set 上以 NDCG/MRR 为目标，而不是只看平均 score。

- 为什么 title 字段可能需要不同 k_1 ？
- 关键词堆砌如何影响 BM25？

不要把 k_1 解释成“关键词权重”；它只调 TF saturation。

k_1 决定“重复多少次以后不再值钱”。

Q27 · BM25 中 b 控制什么？如何理解 $b=0$ 和 $b=1$ ？

难度 3/5 频率 A 适用岗位 IR 调参

30 秒回答

b 控制文档长度归一化强度。 $b = 0$ 完全忽略字段长度； $b = 1$ 使用完整的相对长度校正。

深度拆解

- 相对长度是 $|D|/avgdl$ ，因此不是简单惩罚“绝对长文档”。
- 短字段如 title 的长度差异本来就小，过强 normalization 可能没必要。
- 内容型长文档如果长度差异很大，适当 b 能减少长文偶然匹配优势。
- 不同字段的合理 b 常不同，这也是 BM25F/field-aware scoring 的动机。

- 如果所有文档长度相同， b 还有影响吗？
- 分词粒度改变后 b 的最佳值会变吗？

文档长度是 token 数而不是字节数；analyzer 改动会影响 norm。

b 决定“长文档该被扣多少分”。

Q28 · Title 和 Body 应该如何联合打分？什么是 BM25F 思想？

难度 4/5 频率 A 适用岗位多字段检索

30 秒回答

不同字段的语义和统计分布不同，不能简单拼接成一个大文本。常见方法是 per-field BM25 + boost，或用 BM25F 类方法先对字段内 TF/长度做归一化再融合。

深度拆解

- Title 命中往往比 Body 命中更强，因为 title 是高密度主题字段。
- 字段长度分布不同，应该有不同的 normalization。
- 可构造 $score = w_t BM25_t + w_b BM25_b + \dots$ ，然后用 LTR 学习更复杂融合。
- 字段 boost 不宜纯人工拍脑袋，最好用 judgment/log 数据优化。
- 电商搜索还会加入 brand、category、attribute exact match 等结构化字段。

- 同一个 term 在多个字段出现是否重复计分？
- 字段分词策略可以不同吗？

把字段拼在一起会丢失“在哪里命中”这一重要特征。

多字段搜索的关键是：字段语义不同，统计也不同。

Q29 • BM25 会在哪些场景失败？

难度 3/5 频率 S 适用岗位 IR / Dense

30 秒回答

BM25 的根本限制是 lexical matching。只要 query 与 relevant document 的表面词汇重叠不足，BM25 就可能漏召回；同时它对语义关系、跨语言和复杂意图理解有限。

深度拆解

- Vocabulary mismatch: ‘heart attack’ vs ‘myocardial infarction’。
- 自然语言问题与答案文本用词差异大。
- 短 query 歧义大: ‘apple’、‘jaguar’。
- 跨语言/别名/缩写没有词表扩展时会漏。
- 它也可能被关键词堆砌骗高，需要质量与 spam signals。
- 解决方法不是“抛弃 BM25”，而是 query rewrite、synonym、learned sparse、dense 与 hybrid。

- Dense Retrieval 是否能完全替代 BM25？
- 品牌型号搜索为什么常是 BM25 更强？

语义检索的出现并不意味着 lexical 已过时；两者错误模式互补。

BM25 怕“意思一样、词不一样”。

Q30 · 为什么 BM25 在 2026 年仍然非常强？

难度 3/5 频率 S 适用岗位 IR / Hybrid

30 秒回答

因为 exact match、rare entity、ID/型号、可解释性和成熟倒排执行仍是很多搜索场景的硬需求。Dense 模型补足语义，但常不能稳定替代这些 lexical signals，因此 hybrid 成为更常见方案。

深度拆解

- 零训练成本：没有标注也能获得强基线。
- 倒排索引非常成熟，CPU latency 与内存效率优秀。
- 对罕见实体、代码符号、错误码、SKU 等 OOD/长尾 token 有天然优势。
- 得分可拆解到 term contribution，便于 debug。
- 现代系统常将 BM25 score 作为 LTR 特征，即使主召回已经加入 Dense。

- 如果 embedding 模型很强，BM25 还有什么独有价值？
- Hybrid 如何判断各路的边际收益？

“传统 = 落后”是搜索面试中的危险心态；成熟系统最重视可控、互补与收益/成本比。

BM25 是现代搜索的地基，不是历史遗迹。

参考：R2, R8

第四章 Query Understanding 与 Query Rewrite

搜索效果的上限往往在检索之前就被决定：先理解用户，再谈召回与排序。

题号	题目	难度
Q31	Query Understanding 通常包括哪些任务？	3/5
Q32	中文搜索分词为什么比英文更难？	2/5
Q33	搜索分词是不是越细越好？	3/5
Q34	拼写纠错如何设计 Candidate Generation 与 Ranking？	4/5
Q35	“苹果”这样的 Query 为什么难？如何做意图消歧？	3/5
Q36	什么是 Query Expansion？为什么会同时提升 Recall 和伤害 Precision？	3/5
Q37	Synonym、Query Rewrite、Query Expansion 有什么区别？	2/5
Q38	LLM 如何用于 Query Rewrite？	4/5
Q39	LLM Query Rewrite 最大的风险是什么？	4/5
Q40	如何设计 Search Autocomplete / Query Suggest？	4/5

Q31 · Query Understanding 通常包括哪些任务？

难度 3/5 频率 S 适用岗位 Query / NLP

30 秒回答

Query Understanding 是把用户原始字符串转换成检索系统可消费的结构化意图。典型任务包括 normalization、语言识别、分词、纠错、实体/属性抽取、意图分类、改写与扩展。

深度拆解

- Normalization: 大小写、Unicode、全半角、数字/单位等。
- Segmentation/tokenization: 尤其中文、日文等无天然空格语言。
- Spelling: 纠错与 typo tolerance。
- NER/entity linking: 将“苹果”识别为公司/水果等实体候选。
- Intent/slot: 导航、信息、交易；电商还要抽 brand/category/price/color。
- Rewrite/expansion: 生成更适合 lexical/dense retriever 的查询表示。

- 哪些步骤适合规则，哪些适合模型？
- Query Understanding 的延迟预算通常为什么很紧？
- 如何离线评估 rewrite？

不要把 Query Understanding 等同于“分词”。它是搜索从字符串到意图结构的完整前端。

搜索效果常在“检索之前”就决定了一半。

Q32 · 中文搜索分词为什么比英文更难？

难度 2/5 频率 A 适用岗位中文搜索

30 秒回答

中文没有天然空格边界，而且一个字符串可能有多种合法切分；搜索还要求 recall，往往不能只输出单一分词。

深度拆解

- ‘南京市长江大桥’存在词边界歧义；品牌、网络新词又不断出现。
- 搜索分词与语言学分词目标不同：搜索宁愿保留多个粒度以提高 recall。
- 常见做法是词典 + 统计模型 + 新词发现，并在索引端/查询端做多粒度 token。
- 实体词通常需要整体保留，例如‘北京大学’不能只剩‘北京’与‘大学’。
- 分词策略改变会同时影响 DF、BM25 长度统计和索引大小。

- 索引分词和查询分词应该完全一致吗？
- 新品牌上线后如何快速生效？

搜索分词优化目标不是“切得语言学上最正确”，而是检索质量。

中文搜索要“既切开，也保整体”。

Q33 · 搜索分词是不是越细越好？

难度 3/5 频率 A 适用岗位中文搜索

30 秒回答

不是。过细会丢失实体和短语语义，并显著扩大 **postings**；过粗又会导致召回不足。工业上通常采用多粒度 **token**、短语字段或 **shingles**。

深度拆解

- ‘北京大学’ 若只切为 ‘北京/大学’，会把大量非北大的结果召回。
- 保留原实体 **token** 可提供高 **IDF** 的精确匹配。
- 细粒度 **token** 对 **typo**/部分匹配有帮助，但需要字段权重和最小匹配约束。
- 可将 **exact phrase**、**bigram**、**word token** 作为不同特征交给 **ranker**。

- **N-gram tokenizer** 为什么索引膨胀严重？
- **Edge n-gram** 适合什么场景？

“分得越细 **recall** 越高” 只看到了 **recall**，没有看到 **precision**、**index size** 与实体语义。

粒度是 **recall** 与 **precision** 的旋钮。

Q34 · 拼写纠错如何设计 Candidate Generation 与 Ranking?

难度 4/5 频率 A 适用岗位 Query / NLP

30 秒回答

经典纠错分两步：先用 edit distance、Trie/BK-tree、字典或 confusion set 生成候选，再结合语言模型、词频、上下文和搜索效果排序。现代 LLM 可参与，但应保留“不改”的强候选。

深度拆解

- 候选生成必须高召回：Damerau-Levenshtein 可覆盖插入、删除、替换、交换。
- 候选排序可写成 $score = \alpha edit + \beta freq + \gamma context + \delta retrieval_gain$ 。
- 真实 query 中品牌、人名、型号常是 rare words，盲目纠错会伤害长尾。
- 可利用 query log 学 $typo \rightarrow canonical\ query$ 的共现与点击效果。
- Online 设计常区分 auto-correct 与 “did you mean”，高置信度才自动改。

- 为什么 BK-tree 适合编辑距离搜索？
- 中文拼音纠错与英文 typo 有何不同？
- 如何防止 rare entity 被改坏？

纠错系统的第一目标不是“改得多”，而是“错改率极低”。

纠错要敢于不改。

Q35 · “苹果” 这样的 Query 为什么难？如何做意图消歧？

难度 3/5 频率 A 适用岗位 Intent / Entity

30 秒回答

短 query 信息量低，可能对应多个实体或意图。系统需要结合上下文、用户历史、地域、热度、结果反馈与实体图谱估计 $P(intent | q, u, c)$ 。

深度拆解

- 同一 query 可有 company、fruit、movie/music 等不同实体解释。
- Head query 可从历史点击分布学习 dominant intent；tail query 需要语义模型/知识图谱。
- 结果页可通过 intent diversification 降低一次猜错带来的损失。
- 个性化应谨慎：用户刚搜索食谱时，“苹果”不应因过去看科技新闻就总被解释为 Apple。
- Query session 是很强上下文：前一条 query 往往能显著帮助当前消歧。

- 如何评估 intent classifier？
- 如果两个意图概率接近，结果页怎么排？

不要把“最热门意图”永久固定为唯一答案；搜索需要覆盖 uncertain intents。

短 Query 不是没意图，而是意图不确定。

Q36 · 什么是 Query Expansion? 为什么会同时提升 Recall 和伤害 Precision?

难度 3/5 频率 A 适用岗位 Query Rewrite

30 秒回答

Expansion 为原 query 添加同义、相关实体、缩写展开或伪相关反馈词, 使更多相关文档进入候选; 但扩展词如果语义漂移, 也会召回大量弱相关内容。

深度拆解

- 词典扩展: car → automobile; 实体别名: NYC → New York City。
- Pseudo Relevance Feedback: 从初检 Top docs 中提取扩展词, 例如 RM3 思路。
- Neural/LLM expansion 可以生成语义更丰富的改写, 但漂移风险更大。
- 实践中常给 original terms 更高权重, 扩展词较低 boost。
- 评价应看 recall 增量是否真正转化成 NDCG/CTR, 而不是只看候选数。

- PRF 为什么可能自我强化错误结果?
- Expansion 在 head 与 tail query 哪个更有价值?

Expansion 是 recall 工具, 不应直接被当作“理解正确”的证明。

扩展词要“帮你多找”, 不能“替你改意图”。

Q37 · Synonym、Query Rewrite、Query Expansion 有什么区别？

难度 2/5 频率 A 适用岗位 Query

30 秒回答

Synonym 通常是局部词项等价或近义替换；Expansion 是添加更多检索词；Rewrite 可以重构整个 query，包含删停用信息、纠错、实体规范化、意图补全等更广操作。

深度拆解

- Synonym: ‘iphone ↔ 苹果手机’。
- Expansion: 保留 ‘iphone’，额外加入 ‘Apple phone’ 等候选词。
- Rewrite: ‘北京明天天气怎么样 → 北京天气明天’，可能改变结构。
- 现代搜索常同时生成 lexical rewrite 与 dense rewrite，因为两种 retriever 的理想 query 表达不同。

- Synonym 应在索引时展开还是查询时展开？
- 多路 rewrite 如何融合？

边界并非学术上绝对统一；面试时应说明你采用的工程定义。

Synonym 是局部，Rewrite 是整体，Expansion 是加宽。

Q38 · LLM 如何用于 Query Rewrite?

难度 4/5 频率 A 适用岗位 LLM Search

30 秒回答

LLM 适合把自然语言 query 解析成结构化意图、实体过滤条件和多条检索改写，但必须受 schema、词表和保守策略约束，不能让模型自由“脑补”用户需求。

深度拆解

- 输入可包含原 query、session context、catalog schema、可用 filter/field 说明。
- 输出最好是结构化 JSON: intent、entities、filters、lexical_queries、semantic_queries。
- 对高价值字段做 deterministic validation，例如价格范围、品牌必须来自 catalog。
- 保留 original query 作为一个召回通道，rewrite 只做增量，降低 query drift。
- 低延迟可通过小模型蒸馏、缓存 head queries、异步或只对难 query 启用 LLM。

- 如何检测 LLM rewrite 是否漂移？
- 为什么 dense rewrite 与 BM25 rewrite 可能不同？
- LLM 的成本如何控制？

把 LLM 放到关键路径前，应首先回答延迟、稳定性、可解释性和 fallback。

LLM Rewrite 必须“受约束、可回退、可验证”。

Q39 · LLM Query Rewrite 最大的风险是什么？

难度 4/5 频率 S 适用岗位 LLM Search

30 秒回答

最大的算法风险是 **query drift**：模型补充了用户没有表达的限制或改变了意图；最大的工程风险是延迟/成本和不稳定输出。

深度拆解

- ‘Java GC’ 被改成 ‘Java G1 GC tuning’ 就是过度缩窄。
- 结构化 filter 幻觉更危险：一旦误加 ‘brand=Apple’，后续再强的 ranker 都看不到其他结果。
- 防护: original-query parallel recall、confidence gate、schema validation、rewrite diff logging。
- 需要专门构建“不可改写”样本集：实体、型号、代码、URL、股票代码等。
- 线上监控 query reformulation rate 和 zero-result rate 可发现系统性 drift。

- 如何做 rewrite offline label？
- 是否应该让 LLM 直接输出最终检索结果？

不要用“生成看起来更自然”作为 rewrite 质量标准；要看检索效用和意图保持。

Rewrite 的第一原则：不要改变用户问题。

Q40 · 如何设计 Search Autocomplete / Query Suggest?

难度 4/5 频率 S 适用岗位 Query Suggest / 系统

30 秒回答

Autocomplete 通常是低延迟的候选生成 + 排序系统：Prefix/FST 找候选，结合频率、近期趋势、个性化、拼写和安全策略排序。

深度拆解

- 候选：全局 query log、用户历史、热门趋势、实体/商品词典。
- 前缀索引：Trie/FST/edge n-gram；每个 prefix 可缓存 topN。
- Rank features：prefix match、frequency、recency、CTR、session context、personalization。
- Freshness：热点 query 需要流式更新或单独 hot layer，不能等离线全量 rebuild。
- 安全：敏感、隐私泄露、低频个人 query 不能直接进入全局建议。
- 延迟通常比普通搜索更严格，因为用户每敲一个字符都可能触发请求。

- 如何防止 query log 泄露隐私？
- 趋势词如何避免被刷榜？
- 个性化 suggest 如何与全局候选融合？

Autocomplete 不是“Trie 面试题”，而是一个独立的搜索产品。

Prefix 找候选，**Ranker** 决定你真正看到什么。

第五章 Learning to Rank: 从 RankNet 到 LambdaMART

掌握搜索排序面试的核心数学链：Pointwise → Pairwise → Lambda → Tree-based Ranking。

题号	题目	难度
Q41	为什么需要 Learning to Rank? BM25 不够吗?	3/5
Q42	Pointwise Ranking 是什么? 优缺点?	2/5
Q43	Pairwise Ranking 是什么?	3/5
Q44	Listwise Ranking 是什么? 为什么更贴近 NDCG?	4/5
Q45	RankNet 的核心公式和直觉是什么?	4/5
Q46	LambdaRank 为什么出现? Lambda 到底是什么?	5/5
Q47	LambdaMART 是什么? 为什么经典?	5/5
Q48	深度学习时代, 为什么 LambdaMART 仍然常见?	3/5
Q49	搜索 Ranker 常见特征有哪些? 如何分类?	3/5
Q50	什么是 Query-independent Feature? 为什么有价值?	2/5
Q51	搜索排序中的 Feature Leakage 是什么?	4/5
Q52	Ranker 为什么有时需要 Calibration?	4/5

Q41 · 为什么需要 Learning to Rank? BM25 不够吗?

难度 3/5 频率 S 适用岗位 LTR

30 秒回答

BM25 只利用词项匹配与集合统计,而真实相关性由数十到数千种 query、document、query-document、user 和行为特征共同决定。LTR 用数据学习这些特征如何组合。

深度拆解

- 典型特征: BM25、title exact match、freshness、authority、CTR、dwell、price、quality、embedding sim。
- LTR 的学习对象是“相对排序”,与普通回归/分类的目标不同。
- 经典路线包括 Pointwise、Pairwise、Listwise; 工业经典算法是 LambdaMART。
- 神经模型兴起后, LTR 仍常作为最终 feature fusion 层或蒸馏/重排层。
- 没有稳定 label 和去偏机制, 模型再复杂也只会学习历史排序偏差。

- 为什么不把所有特征直接线性加权?
- LTR 的 training group 为什么以 query 为单位?

“LTR = 用 XGBoost 排序”太窄; 应先讲 ranking learning problem。

BM25 给基础分, LTR 学会“什么时候该信谁”。

参考: R3

Q42 · Pointwise Ranking 是什么？优缺点？

难度 2/5 频率 S 适用岗位 LTR

30 秒回答

Pointwise 把每个 query-document pair 当独立样本，预测 relevance label 或 score。实现简单、可用标准分类/回归，但忽略同一 query 内候选之间的相对关系。

深度拆解

- 输入 $x(q, d)$ ，目标可以是 0/1 relevance 或 0–4 graded relevance。
- 损失可用 MSE、cross entropy 等。
- 最大问题：ranking metric 只关心相对顺序，绝对 label 拟合并不保证好排序。
- 不同 query 的 label scale 还可能不同，pointwise 容易受到 query-level calibration 影响。

- Pointwise 什么时候足够？
- 为什么 score calibration 对 ranking 本身不一定重要？

不要说 pointwise “不能排序”；它当然能用预测 score 排序，只是训练目标没有显式建模相对顺序。

Pointwise 学“分数”，**Pairwise** 学“谁更高”。

Q43 • Pairwise Ranking 是什么？

难度 3/5 频率 S 适用岗位 LTR

30 秒回答

Pairwise 把同一 query 下两个文档构成一对，学习 d_i 是否应排在 d_j 前。RankSVM、RankNet 都是典型方法。

深度拆解

- 将 graded labels 转为偏好对：若 $y_i > y_j$ ，期望 $s_i > s_j$ 。
- 优点是直接对相对顺序建模，不需要 score 在 query 间可比较。
- 缺点是样本对数量可能 $O(n^2)$ ，需要采样。
- 所有 pair 同权仍不理想：把第 1、2 名排反和第 100、101 名排反，对业务影响不同。
- LambdaRank 的核心动机之一就是按 ranking metric 的变化对 pair gradient 加权。

- Pair 数量爆炸怎么采样？
- 只采 hard pair 会有什么风险？

Pairwise 解决了相对性，但还没有天然解决“顶部更重要”。

Pairwise 学偏好，不等于直接优化 NDCG。

Q44 · Listwise Ranking 是什么？为什么更贴近 NDCG？

难度 4/5 频率 A 适用岗位 LTR

30 秒回答

Listwise 直接把一个 query 的候选列表视作训练单元，用整个 permutation/list distribution 构造损失，因此天然更接近最终排序任务，但计算和优化更复杂。

深度拆解

- Pointwise：单文档；Pairwise：文档对；Listwise：整列表。
- ListNet/ListMLE 等方法对排列概率或 top-one probability 建模。
- 真实指标如 NDCG 不平滑、离散，通常仍需 surrogate 或 Lambda 技巧。
- Listwise loss 的 batch/group 构造必须保持 query boundary。
- 候选集合变化会改变 list distribution，因此训练与 serving 候选分布一致很重要。

- Listwise 是否一定比 Pairwise 好？
- 一个 query 候选数不等如何 batch？

“Listwise = 直接对 NDCG 求梯度”不准确；很多 listwise 方法优化的是可微 surrogate。

排序是列表问题，但训练常要用 surrogate 绕开不可微指标。

Q45 · RankNet 的核心公式和直觉是什么？

难度 4/5 频率 A 适用岗位 LTR

30 秒回答

RankNet 为每个文档输出 score，用 score difference 经过 sigmoid 得到一对文档的偏好概率，再用交叉熵训练。

深度拆解

- 若 $s_i = f(x_i), s_j = f(x_j)$ ，则 $P_{ij} = \sigma(s_i - s_j)$ 。
- 如果希望 i 排在 j 前，目标概率为 1。
- 关键不是 sigmoid 本身，而是用 score difference 消除了绝对分值要求。
- 它奠定了 LambdaRank 的 pairwise 梯度框架。

公式 / 复杂度

$$P_{ij} = \frac{1}{1 + \exp(-(s_i - s_j))}, \quad L_{ij} = -\bar{P}_{ij} \log P_{ij} - (1 - \bar{P}_{ij}) \log(1 - P_{ij}).$$

- 为什么只看 score difference?
- 如果所有 score 同时加常数会怎样?

不要死背网络结构；RankNet 的本质是 pairwise logistic loss。

RankNet = 对“谁排前”做 logistic learning。

参考：R3

Q46 · LambdaRank 为什么出现？Lambda 到底是什么？

难度 5/5 频率 S 适用岗位 LTR

30 秒回答

RankNet 的 pair loss 不知道“交换这两个文档会让 NDCG 下降多少”。LambdaRank 不强求构造一个显式可微 ranking loss，而是直接设计梯度（lambda），用 $|\Delta NDCG|$ 等指标变化给 pair 更新加权。

深度拆解

- 若交换顶部两项导致 NDCG 大幅变化，就给予更大梯度。
- 若交换尾部两个低相关项几乎不影响指标，则梯度很小。
- Lambda 既包含 pairwise logistic 的方向/幅度，也乘上 metric sensitivity。
- 这使训练目标更贴近最终排序指标，而无需对 NDCG 本身直接求导。
- 在面试中能说出“metric-weighted gradient”通常已经超过只背名字的候选。

- 为什么 NDCG 不可直接微分？
- Delta NDCG 如何计算？
- LambdaRank 是否保证全局最优？

Lambda 不是某个神秘损失函数名称，而是对每个文档的伪梯度/梯度信号。

LambdaRank：让梯度知道“这次排错有多贵”。

参考：R3

Q47 · LambdaMART 是什么？为什么经典？

难度 5/5 频率 S 适用岗位 LTR

30 秒回答

LambdaMART = LambdaRank 的 ranking-aware lambdas + MART/GBDT 的函数逼近器。它对结构化特征、非线性交互、缺失值和 CPU 推理非常友好，因此长期是搜索排序基线与工业强模型。

深度拆解

- 每轮计算每个样本的 lambda 作为伪残差，回归树去拟合这些梯度。
- 多棵树逐步加法集成： $F_M(x) = \sum_m \eta h_m(x)$ 。
- 树模型擅长处理 ‘BM25 > 阈值且 freshness 很高’ 这类非线性交互。
- 相比深度模型，训练/推理成本低、特征诊断成熟、对中等规模数据鲁棒。
- 现代系统常将 neural similarity 当作一项特征交给 LambdaMART，而不是二选一。

- LambdaMART 与 XGBoost rank:ndcg 有什么关系？
- 为什么树模型不擅长原始文本？
- 如何解释 feature importance？

“LambdaMART 就是 GBDT” 不够；必须指出树拟合的是 ranking lambdas。

LambdaMART = 用树去拟合 NDCG-aware 的梯度。

参考：R3

Q48 · 深度学习时代，为什么 LambdaMART 仍然常见？

难度 3/5 频率 S 适用岗位 LTR / 工业

30 秒回答

搜索最终排序常拥有大量高质量 tabular features，而 GBDT 在这类数据上具有非常强的性价比；神经模型更擅长从文本/图像学习表示，两者组合往往比纯替代更现实。

深度拆解

- 结构化特征包括：BM25、字段命中、质量、年龄、CTR、价格、库存、authority。
- GBDT 对 feature scale 不敏感、缺失处理自然、CPU 延迟稳定。
- 在线 debug 容易：可以解释哪些 feature / tree path 推高了结果。
- Neural ranker 的输出可作为一个 dense relevance feature，加入 GBDT 做最终融合。
- 复杂系统里真正限制模型的是 feature freshness、label bias、serving latency，而不只是表达能力。

- 什么时候应该从 LambdaMART 升级到 DNN？
- 如何做树模型与 Cross-Encoder 融合？

不要把“新模型”与“更好系统”画等号。

文本表示交给神经网络，结构化决策树模型仍很强。

Q49 • 搜索 Ranker 常见特征有哪些？如何分类？

难度 3/5 频率 S 适用岗位 LTR

30 秒回答

最好按 Query、Document、Query-Document、User/Context、Behavior 五类回答，显示你理解特征是否依赖 query、是否可离线预计算，以及是否存在 leakage。

深度拆解

- Query: 长度、语言、频次、意图、是否 head/tail。
- Document: quality、authority、freshness、popularity、长度、类型。
- Query-Document: BM25、字段 exact/phrase match、embedding similarity、entity match。
- User/Context: 地域、设备、历史偏好、session intent。
- Behavior: CTR、CVR、dwell、skip、reformulation，但必须做去偏和时间窗切分。
- 工程上还应标注特征: offline/static、nearline、realtime，以及获取成本。

- 哪些特征可以预计算？
- 实时 CTR 如何做平滑？
- 为什么 query frequency 也可能是强特征？

只列特征名不如说明“特征从哪里来、多久更新、线上怎么取”。

好特征不仅要有预测力，还要能稳定服务。

Q50 · 什么是 Query-independent Feature? 为什么有价值?

难度 2/5 频率 A 适用岗位 LTR

30 秒回答

Query-independent 特征只依赖文档本身,例如 PageRank、质量、freshness、spam score、popularity。它们可离线计算并缓存,对所有 query 复用。

深度拆解

- Query-dependent 特征如 BM25、embedding similarity 必须结合当前 query 计算。
- Query-independent signals 可防止“关键词匹配很好但内容低质”的结果占据顶部。
- 新鲜度通常需要按当前时间动态衰减,因此严格说会依赖 request time,但不依赖 query text。
- 预计算能显著降低 Rank 阶段特征获取成本。

- PageRank 为什么是 query-independent?
- Popularity 会产生什么偏差?

“不依赖 query”不等于“永远静态”; freshness/popularity 仍会随时间变化。

相关性之外,文档本身也有先验质量。

Q51 · 搜索排序中的 Feature Leakage 是什么？

难度 4/5 频率 S 适用岗位 LTR / 数据

30 秒回答

训练时使用了 `serving` 时刻不可获得、或由未来行为计算出的特征，就会产生 `leakage`，使离线指标虚高、上线崩溃。搜索日志尤其容易发生时间穿越。

深度拆解

- 例：用“未来 7 天 CTR”训练今天的样本。
- 例：特征聚合窗口包含了该曝光之后发生的点击。
- 例：人工 `judgment` 只对历史 `Top results` 标注，导致 `selection bias`。
- 解决：`point-in-time join`、`event time watermark`、`feature snapshot/version`、严格 `train/val` 时间切分。
- 训练与 `serving` 还要避免 `feature definition skew`，例如离线 `Python` 逻辑和线上 `C++` 实现不同。

- 随机划分 `train/test` 为什么可能泄漏？
- 同一 `query` 的不同时间样本如何切分？

搜索数据中“时间”不是普通字段，而是防泄漏的第一原则。

任何未来信息都是作弊。

Q52 · Ranker 为什么有时需要 Calibration?

难度 4/5 频率 A 适用岗位 Ranking / 多目标

30 秒回答

纯排序只关心相对顺序，score 的绝对尺度可以任意单调变换；但当 score 被用于阈值、跨模型融合、多目标加权或跨 query 决策时，就需要更稳定的概率/尺度校准。

深度拆解

- 单 query 内排序： s 与 $10s + 100$ 等价。
- 跨通道 score fusion：尺度不同会让某一路永远压制另一路。
- 业务 rule：例如 ‘ $P(CVR) > 0.2$ ’ 需要概率有可解释意义。
- 常见校准：Platt scaling、isotonic regression、temperature scaling。
- 但对 NDCG 最优的 ranking score 未必适合概率校准，需要区分目标。

- Calibration 会不会改变排序？
- 为什么 RRF 不需要 score calibration？

不要为了“看起来像概率”强行校准所有 ranking score。

排序分数用于“排”，概率分数用于“决策”。

CHAPTER 6

第六章搜索指标、点击偏差与实验

会算指标只是起点；高级岗位更看重你能否识别偏差、做因果归因并连接线上结果。

题号	题目	难度
Q53	Precision@K 与 Recall@K 分别衡量什么？	2/5
Q54	MRR 是什么？适合什么场景？	2/5
Q55	MAP 是什么？与 MRR 有何不同？	3/5
Q56	NDCG 是什么？为什么是搜索面试必考？	4/5
Q57	为什么 NDCG 比 Accuracy 更适合 Search？	2/5
Q58	Recall@1000 提升但 NDCG@10 下降，怎么解释？	4/5
Q59	为什么 Click 不能直接当 Relevance Label？	4/5
Q60	如何处理 Position Bias？什么是 IPS？	5/5
Q61	Offline 指标涨了，Online CTR/满意度为什么可能下降？	4/5
Q62	搜索 A/B Test 应如何设计？	4/5

Q53 • Precision@K 与 Recall@K 分别衡量什么？

难度 2/5 频率 S 适用岗位评估

30 秒回答

Precision@K 看前 K 个结果里有多少是相关的；Recall@K 看所有相关结果中有多少被前 K 个找到了。召回阶段更关心 Recall，用户可见顶部排序更关心 Precision/NDCG。

深度拆解

- $P@K = \#relevant@K / K$ 。
- $R@K = \#relevant@K / \#all\ relevant$ 。
- 当 total relevant 不可完整标注时，Recall 的估计会受 judgment pool 影响。
- RAG retrieval 中 Recall@K 常非常关键，因为漏掉证据后 generator 无法恢复。

- Precision 与 Recall 能同时无限提高吗？
- 为什么搜索常不用单一 F1？

搜索是排名问题，仅 P/R 还没考虑相关结果出现的位置。

Precision 看“前面干不干净”，**Recall** 看“该找的漏没漏”。

Q54 · MRR 是什么？适合什么场景？

难度 2/5 频率 S 适用岗位评估

30 秒回答

MRR 只关心第一个 relevant result 的排名，因此非常适合导航搜索、问答、实体查找等“找到一个正确答案就基本完成任务”的场景。

深度拆解

- 若第一个相关结果排名为 r_q ，则 $RR_q = 1/r_q$ 。
- $MRR = \frac{1}{|Q|} \sum_q RR_q$ 。
- 第二个、第三个相关结果不会继续贡献，因此不适合需要多个相关结果的 broad search。
- MRR 对顶部位置极其敏感。

公式 / 复杂度

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}.$$

- 如果第一个 relevant 在 rank1 与 rank2，差多少？
- 电商搜索为什么通常不能只用 MRR？

记住 MRR 的局限：只看“第一个”。

MRR = First Relevant 的倒数排名。

Q55 · MAP 是什么？与 MRR 有何不同？

难度 3/5 频率 A 适用岗位评估

30 秒回答

Average Precision 在每个 relevant result 出现的位置计算 Precision，并对这些值求平均；MAP 再对 query 求均值。因此它会考虑多个 relevant results，而不是只看第一个。

深度拆解

- $AP = \frac{1}{R} \sum_k P@k \cdot rel(k)$ 。
- 如果相关文档都集中在前面，AP 高。
- MAP 适合 binary relevance；graded relevance 通常更偏向 NDCG。
- Incomplete judgments 会影响 MAP，尤其尾部未判文档。

- MAP 与 NDCG 谁更适合 graded labels？
- 为什么 AP 的分母是 relevant 总数？

不要把 MAP 误解为 mean precision@K。

MAP 奖励 “所有相关结果都尽早出现”。

Q56 · NDCG 是什么？为什么是搜索面试必考？

难度 4/5 频率 S 适用岗位评估 / LTR

30 秒回答

NDCG 同时表达“高相关等级更值钱”和“越靠前越值钱”，再用理想排序归一化到可跨 query 比较的尺度，因此非常适合 graded relevance 的 Web/电商搜索。

深度拆解

- Gain 常用 $2^{rel_i} - 1$ ，让 3→4 的提升比 0→1 更重要。
- Discount 常用 $1/\log_2(i+1)$ ，体现顶部位置价值更高。
- $IDCG@K$ 是把同一 query 的 labels 理想排序后的 DCG。
- $NDCG = DCG/IDCG$ ，理想排序为 1。
- LambdaRank/LambdaMART 常用 $\Delta NDCG$ 衡量 pair swap 的重要度。

公式 / 复杂度

$$DCG@K = \sum_{i=1}^K \frac{2^{rel_i} - 1}{\log_2(i+1)}, \quad NDCG@K = \frac{DCG@K}{IDCG@K}.$$

- 为什么要归一化？
- Gain 为什么常用指数而不是直接 rel？
- $IDCG=0$ 怎么处理？

面试时不要只写公式；必须讲清 gain、discount、normalization 三个设计选择。

NDCG = “价值” × “位置折扣” ÷ “该 query 的理想上限”。

参考：R1

Q57 · 为什么 NDCG 比 Accuracy 更适合 Search?

难度 2/5 频率 A 适用岗位评估

30 秒回答

Accuracy 把每个样本当独立分类，无法区分“相关结果排第 1”和“排第 100”的巨大体验差异，也无法自然处理 graded relevance。

深度拆解

- 搜索输出是 ordered list，不是单标签分类。
- 顶部位置暴露和用户注意力远高于尾部，因此 position discount 必要。
- 一个系统可能预测相关/不相关准确率很高，但把少数真正重要的相关结果全排在后面。
- 因此 ranking metrics 必须评价顺序而非仅标签预测。

- ROC-AUC 适合排序吗？
- 为什么线上 CTR 也不是 NDCG 的替代品？

分类指标可以辅助诊断，但不能替代 list-level ranking metric。

搜索错一位和错一百位，不是同一种错误。

Q58 • Recall@1000 提升但 NDCG@10 下降，怎么解释？

难度 4/5 频率 S 适用岗位诊断

30 秒回答

说明“候选更全”并不代表“顶部更好”。新召回通道可能引入大量 hard negatives，改变 Ranker 的候选分布；也可能 Ranker 没学会利用新通道，或候选预算挤掉了原来高质量候选。

深度拆解

- 先做 oracle analysis：在新候选集合上，用真实 label 的理想排序看 NDCG 上限是否提高。
- 分析 relevant docs 是否确实更多进入 candidate set。
- 检查 Ranker training 是否包含新召回分布；若没有，会发生 distribution shift。
- 按 recall source 打 feature、score 和 topK occupancy，定位谁挤占了位置。
- 必要时对 source 做 quota、fusion 或重新训练 ranker。

- Recall 增量为什么可能是“无效 recall”？
- 如何做 channel ablation？

一个阶段指标上升不等于 end-to-end 上升；搜索必须按 cascade 做归因。

召回决定上限，排序决定你能拿到多少上限。

Q59 · 为什么 Click 不能直接当 Relevance Label?

难度 4/5 频率 S 适用岗位 Click / LTR

30 秒回答

用户只会点击被展示且被看到的结果，所以 click 同时受 relevance 与 examination 影响。历史排名制造了曝光偏差，直接训练会把旧模型偏差复制到新模型。

深度拆解

- Position bias: rank1 获得更多 exposure。
- Selection bias: 未进入历史 topK 的文档永远没有点击 label。
- trust/presentation bias: 品牌与视觉样式影响点击。
- 点击还包含 attractiveness, 而不纯是任务满意度。
- 可加入 dwell、return-to-SERP、reformulation 等后续行为改善 label。

- 未点击结果能否作为负样本?
- Skip-above click 能说明什么?

“曝光未点击 = 不相关” 同样不成立，尤其用户可能根本没看到该结果。

点击日志是观察数据，不是随机实验。

参考: R4

Q60 · 如何处理 Position Bias? 什么是 IPS?

难度 5/5 频率 S 适用岗位 Unbiased LTR

30 秒回答

核心思想是估计每个位置被 examination 的 propensity，并用其倒数对观察到的点击重新加权，使高位点击不被系统性高估。随机化/intervention 常用于估计 propensity。

深度拆解

- 简化点击模型： $P(C = 1) = P(E = 1 \mid position)P(R = 1 \mid q, d)$ 。
- IPS 风险估计对样本乘 $1/p_i$ ；低 propensity 位置的点击权重更大。
- Propensity 很小时权重会爆炸，需要 clipping/self-normalization 等方差控制。
- Interleaving/random swaps 能获得更可靠的相对偏好，但要控制用户体验成本。
- Unbiased LTR 的难点从“拟合模型”转向“识别曝光机制”。

公式 / 复杂度

$$\hat{R}_{IPS} = \frac{1}{n} \sum_{i=1}^n \frac{c_i}{p_i} \ell_i.$$

- IPS 为什么方差大?
- 如果 propensity model 错了会怎样?
- 如何做 doubly robust?

不要只说“随机打乱位置”；面试要说明为什么随机化能估计 examination propensity。

IPS = 用“被看到的概率”纠正历史曝光。

参考: R4

Q61 · Offline 指标涨了，Online CTR/满意度为什么可能下降？

难度 4/5 频率 S 适用岗位实验

30 秒回答

离线 judgment/log 数据只是线上用户分布的近似，而且不包含延迟、展示、系统交互与长期行为。离线改进可能过拟合 label 或损害未被评价的目标。

深度拆解

- Label mismatch: 人工 relevance 与用户点击/转化目标不同。
- Distribution shift: 评测集 head query 过多，线上 tail query 更多。
- Latency regression: 模型相关性更好，但 p99 增加导致用户流失。
- Diversity/freshness 未纳入离线指标。
- 日志本身有历史系统 selection bias。
- 局部指标提升可能伤害长期指标，例如短点击变多、long click 下降。

- 如何设计 offline-online correlation study?
- 为什么 slice metrics 很重要?

Offline 是筛选器，不是最终裁判。

离线告诉你“值得上线试”，A/B 才告诉你“用户是否更好”。

Q62 · 搜索 A/B Test 应如何设计？

难度 4/5 频率 S 适用岗位实验 / 产品

30 秒回答

先明确主指标与 guardrails，再随机分流、控制样本污染与实验时间，并做 query/user slice。搜索实验尤其要关注 latency、zero-result、abandonment 等防线。

深度拆解

- Primary: CTR、successful search、conversion、long-click、task completion。
- Guardrails: p95/p99 latency、error、zero result、投诉、reformulation、revenue risk。
- 随机化单位可选 user/request/query，必须与干预的干扰范围匹配。
- Head/tail、new/returning、locale、device、query intent 分 slice。
- 避免 novelty effect、seasonality、sample ratio mismatch。
- 实验结论要同时报告 effect size、置信区间与 practical significance。

- 为什么 user-level randomization 常比 request-level 稳？
- 搜索 interleaving 和 A/B 有何差异？

只报 p-value 不够；业务上 0.01

A/B = 效果 + 风险 + 成本的线上验收。

第七章 Dense Retrieval 与 ANN

从对比学习到十亿向量索引，掌握 Dual Encoder、Hard Negative、IVF、PQ、HNSW 的完整链路。

题号	题目	难度
Q63	Dense Retrieval 与 BM25 的本质区别是什么？	3/5
Q64	为什么 Dual Encoder 适合召回？	3/5
Q65	双塔检索模型通常怎么训练？	4/5
Q66	为什么 Hard Negative 对 Dense Retrieval 至关重要？	4/5
Q67	什么是 In-batch Negative？有什么坑？	3/5
Q68	Cosine、Inner Product、L2 在归一化向量下有什么关系？	3/5
Q69	为什么十亿向量不能直接暴力扫描？如何算量？	3/5
Q70	IVF 的原理是什么？nlist 与 nprobe 如何影响效果？	4/5
Q71	Product Quantization (PQ) 是什么？	5/5
Q72	HNSW 的原理是什么？为什么分层？	5/5
Q73	HNSW 的 M、efConstruction、efSearch 分别控制什么？	4/5
Q74	HNSW 与 IVF-PQ 如何选？	5/5

Q63 · Dense Retrieval 与 BM25 的本质区别是什么？

难度 3/5 频率 S 适用岗位 Dense Retrieval

30 秒回答

BM25 在词项空间做稀疏 lexical matching; Dense Retrieval 把 query/document 编码为低维向量, 通过内积、cosine 或 L2 找语义邻居。它解决 vocabulary mismatch, 但会失去部分 exact sparse 信号。

深度拆解

- BM25 的每个维度对应词表 term; dense embedding 的维度没有直接词义解释。
- Dense 可把表面词不同但语义接近的文本拉近。
- Dense 对实体、数字、型号等 exact matching 可能不如 BM25 稳。
- Dense 检索需要 ANN index; BM25 依赖 inverted index。
- 两者互补是 hybrid search 成为主流的重要原因。

- Dense 是否还需要 tokenizer?
- 为什么 embedding normalization 很重要?

不要说 Dense “理解语义所以总比 BM25 好”; 它的错误模式不同。

BM25 比词, Dense 比表示。

Q64 · 为什么 Dual Encoder 适合召回？

难度 3/5 频率 S 适用岗位 Dense Retrieval

30 秒回答

Dual Encoder 独立编码 query 和 document，使 document embedding 可以离线预计算并建立 ANN 索引；在线只算一次 query embedding，再进行向量检索。

深度拆解

- $q = E_q(query)$, $d = E_d(doc)$, score 常为 $q^T d$ 。
- 文档向量可复用给所有 query，这是其相对 Cross-Encoder 的决定性效率优势。
- 代价是 query-document token 不能在 Transformer 内部做全交互，表达能力受限。
- 因此常用于第一阶段 recall，再用 Cross-Encoder rerank。

- Query encoder 和 Doc encoder 是否共享参数？
- 为什么 asymmetric retrieval 常不共享？

Dual Encoder 的价值不只是“快”，而是把在线 pair computation 变成可索引表示。

可预计算，才可全库 ANN。

Q65 · 双塔检索模型通常怎么训练？

难度 4/5 频率 S 适用岗位 Dense Training

30 秒回答

最常见是 contrastive learning：让 query 与正文档相似度高于一组负文档。InfoNCE/softmax loss 将检索训练转成 batch 内分类问题。

深度拆解

- 正样本可来自点击、人工 relevance、QA evidence、同文档标题-正文等。
- 负样本可由 random、BM25 hard negative、model-mined negative 混合。
- Temperature τ 控制 softmax 分布尖锐度。
- 如果使用 in-batch negatives, batch size 直接影响负样本数量。
- 训练后要用真实 ANN 而非只用 embedding exact search 评估 serving recall。

公式 / 复杂度

$$L = -\log \frac{\exp(s(q, d^+)/\tau)}{\exp(s(q, d^+)/\tau) + \sum_j \exp(s(q, d_j^-)/\tau)}.$$

- False negative 对 contrastive loss 有何影响？
- 为什么 hard-negative mining 要迭代？

正负样本质量通常比“换一个更大 backbone”更能决定 dense retriever 上限。

Dense Retriever 的训练本质：把正样本拉近，把有价值的负样本推远。

Q66 · 为什么 Hard Negative 对 Dense Retrieval 至关重要？

难度 4/5 频率 S 适用岗位 Dense Training

30 秒回答

随机负样本通常与 query 完全无关，模型很快学会区分，梯度趋于无信息。Hard negative 在主题/词面上很像正确答案却不真正 relevant，迫使模型学习细粒度语义边界。

深度拆解

- 经典做法：BM25 top results 中排除正样本，作为 hard negatives。
- 也可用当前 dense model 自己挖 top false positives，进行 iterative mining。
- Too-hard negative 可能其实是未标注 relevant (false negative)，会伤害训练。
- 应混合 easy + medium + hard negatives，保持优化稳定。
- 负样本分布最好接近 serving candidate distribution。

- BM25 negative 为什么特别有价值？
- 如何发现 false negatives？

“越难越好”是错误的；错误标注的超难负样本会把真正相关文档推远。

Hard negative 要难，但必须真的负。

Q67 · 什么是 In-batch Negative? 有什么坑?

难度 3/5 频率 S 适用岗位 Dense Training

30 秒回答

一个 batch 中每个 query 的正样本文档, 可被其他 query 当作负样本, 从而无需额外编码大量 negatives。

深度拆解

- Batch 中 B 个 pair, 可为每个 query 提供约 $B - 1$ 个 negatives。
- 矩阵乘法一次计算 $B \times B$ 相似度, GPU 利用率高。
- 坑: batch 内可能出现语义相关或真实 relevant 文档, 形成 false negative。
- Distributed training 可做 cross-device negatives, 显著扩大负样本池, 但通信成本增加。
- 数据去重和 query grouping 很重要, 避免同一主题样本相互错误排斥。

- Batch size 越大一定越好吗?
- Cross-device negatives 如何实现?

In-batch negative 省编码, 不代表 label 自动正确。

用 batch 规模“免费”扩大负样本池。

Q68 • Cosine、Inner Product、L2 在归一化向量下有什么关系？

难度 3/5 频率 A 适用岗位 ANN 数学

30 秒回答

若向量都做 L2 normalization, 则 cosine similarity 就是 inner product, 而平方 L2 距离 $\|x - y\|^2 = 2 - 2x^T y$, 三者给出相同排序。

深度拆解

- 未归一化时 inner product 同时受方向与向量模长影响。
- 有些 retriever 训练本来就利用 norm 表达置信度, 这时不能随意 normalize。
- Faiss 做 cosine 常采用 normalize vectors + IndexFlatIP/inner-product index。
- ANN 索引的 metric 必须与训练时的 similarity 定义一致。

公式 / 复杂度

$$\|x - y\|_2^2 = \|x\|^2 + \|y\|^2 - 2x^T y = 2 - 2x^T y \quad (\|x\| = \|y\| = 1).$$

- 为什么 MIPS 不是普通 Euclidean NN?
- 归一化是否会改变训练模型效果?

不要在模型训练后无脑把所有向量归一化; 先确认训练目标。

Unit norm 下: cosine = dot, L2 只是单调变换。

参考: R6

Q69 · 为什么十亿向量不能直接暴力扫描？如何算量？

难度 3/5 频率 S 适用岗位 ANN / 系统

30 秒回答

Exact Flat Search 的每次 query 需要对 N 个 d 维向量计算距离，复杂度 $O(Nd)$ ，同时还要从内存读取整个向量库。十亿级时通常受内存带宽和算力双重限制。

深度拆解

- 例如 $1\text{B} \times 768\text{-d float32}$ ，仅向量原始存储约 $10^9 \times 768 \times 4 \approx 3.07\text{ TB}$ 。
- 每次 query 全扫 3 TB 数据即使向量点积很快也不现实。
- ANN 的核心是只访问一个小子集：聚类分桶（IVF）或图搜索（HNSW）。
- 另一维是压缩：PQ/SQ/OPQ 将每个向量从 KB 降到几十字节。
- 系统设计要同时算 storage、QPS、latency、recall 与 build/update cost。

- GPU Flat Search 什么时候反而可行？
- 1M 与 1B 向量索引选择有何不同？

只说“ $O(N)$ 太慢”不够，高级面试应能做 memory bandwidth 的数量级估算。

ANN 的起点是算清楚“全扫要读多少数据”。

参考：R6

Q70 · IVF 的原理是什么？nlist 与 nprobe 如何影响效果？

难度 4/5 频率 S 适用岗位 ANN / Faiss

30 秒回答

IVF 先用 coarse quantizer 把向量空间划分成多个倒排 cell；查询只访问与 query 最近的若干 cell。‘nlist’ 决定分区粒度，‘nprobe’ 决定一次搜索多少个 cell。

深度拆解

- 建库：训练 k-means centroids，将每个 base vector 分配到最近 centroid。
- 查询：先找 nearest centroids，再扫描这些 inverted lists 内的向量或 compressed codes。
- $nprobe \uparrow \Rightarrow recall \uparrow, latency \uparrow$ 。
- ‘nlist’ 太小每个 list 太长，搜索慢；太大则训练/路由成本上升且 cell 可能不均衡。
- IVF 与 PQ 常组合成 IVFPQ，同时减少访问数量与每向量存储。

- 数据分布漂移后 centroids 会怎样？
- 为什么需要 representative training sample？

IVF 的“倒排”与文本倒排思想相似：先定位少量 bucket，再局部扫描。

IVF = 向量空间的 coarse partition + selective scan。

参考：R6

Q71 • Product Quantization (PQ) 是什么?

难度 5/5 频率 S 适用岗位 ANN / 压缩

30 秒回答

PQ 将 d 维向量切成 m 个低维子向量，每个子空间独立学习码本，只存每个子向量对应的 code。距离可用 lookup table 在压缩域近似计算。

深度拆解

- 若每个子量化器 256 个中心，只需 8 bit 表示一个 code。
- $m = 16$ 时一个向量可压到 16 bytes（不含 id），远小于 768-d float32 的 3072 bytes。
- 查询时对每个子空间预计算 query 到 codewords 的距离表，再累加 code lookup。
- PQ 引入 quantization error，代码越短压缩越强但 recall 降低。
- OPQ 在 PQ 前学习旋转/线性变换，使子空间更适合量化。

- 为什么 PQ 要分子空间?
- IVFPQ 为什么通常对 residual 做 PQ?
- ADC 是什么?

PQ 不是把 float32 简单变 int8；它是学习式向量量化。

PQ = 用多个小码本组合表示一个高维向量。

参考: R6

Q72 · HNSW 的原理是什么？为什么分层？

难度 5/5 频率 S 适用岗位 ANN / HNSW

30 秒回答

HNSW 构建多层 proximity graph: 上层节点少、连接跨度大, 用于快速粗粒度导航; 逐层下降到更密的底层图做局部精细搜索, 类似 skip list 的多尺度跳跃。

深度拆解

- 每个向量随机分配最高层, 层数概率指数衰减, 因此高层非常稀疏。
- 搜索从顶层入口开始 greedy navigation, 再到下一层, 以当前最好节点为入口。
- 底层使用候选队列扩展邻居, 'efSearch' 控制探索宽度。
- 建图时也要搜索邻居并做 heuristic neighbor selection, 'efConstruction' 控制质量。
- 图方法在高 recall 区域通常非常强, 但 graph links 带来显著内存开销。

- HNSW 为什么不是严格 $O(\log N)$ 保证?
- 删除向量为什么麻烦?
- 入口点如何选择?

不要把 HNSW 描述成“多层 KNN 图”就结束; 要解释层级带来的 long-range navigation。

HNSW = 用稀疏上层快速靠近, 再在底层精搜。

参考: R5, R6

Q73 · HNSW 的 M、efConstruction、efSearch 分别控制什么？

难度 4/5 频率 S 适用岗位 ANN 调参

30 秒回答

‘M’ 决定图的连接度与内存；‘efConstruction’ 决定建图时搜索宽度和图质量；‘efSearch’ 决定查询时探索宽度，是 recall-latency 最直接的在线旋钮。

深度拆解

- ‘M’ 大：图更密、recall 通常更高、memory/index build cost 更高。
- ‘efConstruction’ 大：建图更慢，但图连通和近邻质量更好。
- ‘efSearch’ 大：查询访问更多节点，recall 更高、latency 更大。
- 经验参数不能脱离数据规模、维度、metric 和 QPS；必须跑 recall-latency curve。
- Faiss 当前文档明确把 M、efConstruction、efSearch 列为 HNSW 关键参数。

- M 提高后 efSearch 是否可以降低？
- 为什么建库成本与查询成本可以独立调？

最常见面试错误：把 efConstruction 和 efSearch 说反。

M 管结构，**efConstruction** 管建图，**efSearch** 管查询。

参考：R6

Q74 · HNSW 与 IVF-PQ 如何选?

难度 5/5 频率 S 适用岗位 ANN 系统设计

30 秒回答

选择取决于内存预算、规模、更新模式、目标 recall、CPU/GPU 和 build cost。HNSW 通常高 recall、低延迟但内存重；IVF-PQ 压缩强、适合超大规模和 GPU，但有训练与量化误差。

深度拆解

- HNSW：不依赖 k-means training，在线 recall-latency 好；graph overhead 可达每向量数百字节量级。
- IVFPQ：向量可压到几十 bytes，适合百亿/十亿级内存约束。
- 频繁更新场景：HNSW 的删除/重建管理较复杂；IVF 也要处理 centroid drift 与 tombstone。
- GPU：Faiss IVF/PQ 路线非常成熟；HNSW 更常见于 CPU 内存索引。
- 最佳答案不是“哪个更强”，而是画出 recall/latency/memory/build/update 五维 trade-off。

- 1B 768-d、128GB RAM 怎么选？
- 如果要求 99.9
- 如果要求每日全量重建呢？

任何不带约束的“选 HNSW/IVF”都是不完整答案。

索引选择是多约束优化，不是算法排行榜。

参考：R6

第八章 Hybrid Search、Neural ReRank 与 RAG

把 BM25、Dense、Learned Sparse、Cross-Encoder、ColBERT 与 Agentic Retrieval 放到一条现代检索栈里理解。

题号	题目	难度
Q75	为什么 Hybrid Search 往往比纯 BM25 或纯 Dense 更稳?	3/5
Q76	BM25 Score 与 Dense Cosine 能直接相加吗?	3/5
Q77	什么是 Reciprocal Rank Fusion (RRF)?	3/5
Q78	SPLADE 这类 Learned Sparse Retrieval 在做什么?	5/5
Q79	为什么 Cross-Encoder 适合 Rerank, 而不适合全库 Recall?	3/5
Q80	Dual Encoder 与 Cross-Encoder 的经典 Trade-off 是什么?	2/5
Q81	ColBERT 的 Late Interaction 为什么重要?	5/5
Q82	RAG 的 Chunk Size 应该怎么选?	4/5
Q83	Retrieval Recall 很高, 为什么 RAG 仍会答错?	4/5
Q84	什么是 Agentic / Iterative Search?	5/5

Q75 · 为什么 Hybrid Search 往往比纯 BM25 或纯 Dense 更稳？

难度 3/5 频率 S 适用岗位 Hybrid

30 秒回答

Lexical 与 Dense 的失败模式互补：BM25 擅长 exact/rare terms，Dense 擅长同义改写与语义匹配。把两路候选并集并融合，能提高 robustness，尤其对真实 query 的长尾分布。

深度拆解

- BM25 强：SKU、错误码、人名、代码 token、短语精确命中。
- Dense 强：自然语言问题、同义表达、词汇错配。
- Hybrid 可在 recall 层做 union，也可在 rank 层把两种 score/features 交给 LTR。
- 要做 channel contribution 分析，避免两个通道高度重复却白白增加成本。
- 对 head/tail、entity/non-entity query 分 slice，常能看到明显互补模式。

- Hybrid 一定能提升吗？
- 什么时候 dense 通道应该动态关闭？

简单 union 会增加下游负担；hybrid 的价值需要通过融合和 ranker 真正兑现。

BM25 找“同词”，Dense 找“同义”；Hybrid 要两者都保。

Q76 • BM25 Score 与 Dense Cosine 能直接相加吗？

难度 3/5 频率 S 适用岗位 Hybrid Fusion

30 秒回答

通常不能，因为两个分数的尺度、分布和 query 依赖性不同。需要 normalization、learned fusion，或完全绕过 score、只使用 rank 的 RRF。

深度拆解

- BM25 score 未归一且跨 query 不可直接比较；cosine 常在 $[-1,1]$ 或更窄区间。
- Min-max/z-score 受 query candidate distribution 影响。
- Learned fusion 可把 BM25、dense score、rank、query type 交给 LTR。
- RRF 只依赖 rank，避免 score calibration。
- 最佳权重常随 query intent 变化：entity query 更信 lexical，natural-language question 更信 dense。

- 为什么全局 min-max 可能不稳定？
- 如何学习动态融合权重？

“归一化后相加”不是总能解决问题；分数的语义本身可能不同。

先问 **score** 是否可比，再谈相加。

Q77 · 什么是 Reciprocal Rank Fusion (RRF)?

难度 3/5 频率 S 适用岗位 Hybrid Fusion

30 秒回答

RRF 用文档在各个结果列表中的名次，而不是原始 score，计算 reciprocal-rank 加和。它简单、无需训练、对不同 score scale 鲁棒，因此是 hybrid search 常见 baseline。

深度拆解

- 若文档在第 $r_i(d)$ 名，则贡献 $1/(k + r_i(d))$ 。
- k 是平滑常数，降低 top rank 极端差异。
- 某文档被多路检索都排得较高，会得到累积优势。
- RRF 不需要知道 BM25 20 分与 cosine 0.82 哪个“更大”。
- 缺点是丢掉了原 score magnitude，且通道权重默认相同，需要扩展时可加权。

公式 / 复杂度

$$RRF(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + r(d)}.$$

- RRF 的 k 怎么影响融合？
- Weighted RRF 怎么做？
- 为什么 rank-based fusion 可能损失信息？

RRF 很强但不是万能：它牺牲 score 信息换鲁棒性。

RRF = 不比原始分，只比“各路排第几”。

参考：R9

Q78 · SPLADE 这类 Learned Sparse Retrieval 在做什么？

难度 5/5 频率 A 适用岗位 Neural IR

30 秒回答

SPLADE 使用 Transformer 学习稀疏词表权重与 expansion，使文档/query 仍可通过倒排索引检索，同时获得神经模型的语义扩展能力。

深度拆解

- 输出维度对应词表，而不是一个低维 dense vector。
- 模型可以给输入中没出现的词分配权重，实现 learned expansion。
- 稀疏正则/效率约束用于控制激活 term 数量，否则倒排索引会爆炸。
- 优势：仍支持 inverted index、exact lexical signal 与一定可解释性。
- 它位于 BM25 与 Dense Retrieval 之间，是“神经化稀疏检索”的代表。

- SPLADE 为什么需要 sparsity regularization?
- Learned expansion 会产生什么奇怪 token?

不要把 SPLADE 说成“BERT 生成同义词”；它输出的是词表维度上的可学习 sparse weights。

SPLADE = 神经模型学会“哪些词该被激活、权重多大”。

参考：R8

Q79 · 为什么 Cross-Encoder 适合 Rerank，而不适合全库 Recall?

难度 3/5 频率 S 适用岗位 Neural Ranking

30 秒回答

Cross-Encoder 把 query 和 document 一起输入 Transformer，token 级交互充分，相关性效果强；但每个 query-document pair 都要单独前向，无法预计算文档向量，所以只适合小候选集。

深度拆解

- 输入类似 '[CLS] query [SEP] doc'，self-attention 能建模细粒度匹配。
- 候选数 $K=100$ 时做 100 次 pair inference；全库十亿文档显然不可能。
- 常在 BM25/Dense top100-1000 后做 rerank。
- 可用 distillation 将 Cross-Encoder 知识蒸馏给 Dual Encoder/PreRank。
- 长文档需要截断、passage aggregation 或 hierarchical scoring。

- Cross-Encoder 的复杂度如何随 doc length 增长？
- 如何批量化降低 GPU latency？

Cross-Encoder 的最大优势和最大代价来自同一件事：query/doc 在模型内完全交互。

交互越深，越难预计算。

Q80 · Dual Encoder 与 Cross-Encoder 的经典 Trade-off 是什么？

难度 2/5 频率 S 适用岗位 Neural Ranking

30 秒回答

Dual Encoder 牺牲 token-level 交互换可索引、低延迟；Cross-Encoder 牺牲吞吐换更强的 pairwise relevance。工业 cascade 通常让它们分工，而不是二选一。

深度拆解

- Dual: doc embedding 可离线、ANN 支持、适合 recall。
- Cross: pair input、无法预计算、适合 rerank。
- Dual 的表示瓶颈是把整段压成一个或少数向量。
- Cross 的瓶颈是 K 倍模型前向与长上下文成本。

- ColBERT 为什么被称为二者之间？
- Bi-encoder 能否通过多向量缓解瓶颈？

如果回答只说“一个快一个准”，要继续解释为什么。

可索引性与交互能力是一对核心矛盾。

Q81 · ColBERT 的 Late Interaction 为什么重要？

难度 5/5 频率 A 适用岗位 Neural IR

30 秒回答

ColBERT 独立编码 query/document 的 token embeddings，让文档表示可离线；查询时再做轻量 MaxSim late interaction，保留比单向量 bi-encoder 更细粒度的 token 对齐。

深度拆解

- 每个 query token 找最相似的 document token，再求和。
- 相较 Cross-Encoder，昂贵 BERT 编码可对文档离线完成。
- 相较单向量 Dual Encoder，不必把文档压成一个向量，因此细粒度匹配更强。
- 代价是一个文档要保存多 token vectors，索引和存储远高于单向量。
- 现代 ColBERT 系列通过压缩、残差和专用索引进一步降低成本。

公式 / 复杂度

$$S(q, d) = \sum_{i \in q} \max_{j \in d} E_q(q_i)^T E_d(d_j).$$

- MaxSim 为什么不是平均池化？
- ColBERT 文档索引为什么更大？

Late Interaction 的重点是“编码早分离，匹配晚发生”。

ColBERT = 可预计算的 token-level interaction。

参考：R7

Q82 · RAG 的 Chunk Size 应该怎么选?

难度 4/5 频率 S 适用岗位 RAG Retrieval

30 秒回答

没有通用最优值。Chunk 太小会丢上下文、跨句关系和表格结构；太大则 embedding 主题混杂、检索精度下降，并浪费生成上下文。需要结合文档结构、retriever、reranker 与最终 QA 做端到端选择。

深度拆解

- 优先结构感知 chunk: heading/paragraph/table/code block, 而不是固定字符硬切。
- Overlap 能缓解边界信息丢失, 但会制造重复候选和 token 浪费。
- 可使用 parent-child retrieval: 小 chunk 检索, 返回更大的 parent context。
- 对事实型 QA 与长篇解释型 QA, 最佳 chunk 粒度通常不同。
- 评价至少要同时看 retrieval recall、context precision、answer quality、latency/token cost。

- Overlap 设太大有什么问题?
- 表格应该如何 chunk?
- 是否可以 query-aware dynamic chunking?

“512 tokens 是最佳实践”不是结论, 只是常见起点。

Chunk size 是检索与生成共同的超参数。

Q83 • Retrieval Recall 很高，为什么 RAG 仍会答错？

难度 4/5 频率 S 适用岗位 RAG 诊断

30 秒回答

因为“证据被召回”只是必要条件。Reranker 可能没把证据放进最终 context，证据可能被冗余 chunk 淹没，生成模型也可能没有正确利用或推理。

深度拆解

- Candidate recall 高，但 final context recall 低：问题在 rerank/context builder。
- 证据存在但位置太靠后：可能出现 lost-in-context。
- 多跳问题只召回一半证据，单文档 relevance 指标看起来仍不错。
- 生成模型可能 hallucinate 或忽略证据，需要 faithfulness/attribution 评估。
- 诊断要分层：Retriever → Reranker → Context → Generator，而不是只看最终准确率。

- 如何做 oracle generator 分析？
- 如何区分 retrieval error 与 reasoning error？

RAG 评估必须可归因，否则只调 embedding 很容易陷入盲目优化。

证据“在候选里”不等于模型“真正用到了”。

Q84 · 什么是 Agentic / Iterative Search?

难度 5/5 频率 A 适用岗位 LLM Search

30 秒回答

传统 retrieval 一次 query 后就固定结果；Agentic Search 允许模型根据当前证据判断缺口、生成下一轮 query、执行多跳检索和 sufficiency check，直到证据足够或预算耗尽。

深度拆解

- Plan：把复杂问题拆成子问题。
- Retrieve：针对子问题检索。
- Inspect：抽取证据并发现缺口。
- Rewrite/Next-hop：根据已知信息生成下一步 query。
- Sufficiency：判断证据是否足够，避免无限循环。
- 关键工程问题：tool latency、query explosion、重复检索、引用追踪与预算控制。

- 如何定义停止条件？
- 如何防止 agent 被错误证据带偏？
- 多跳检索如何去重和缓存？

Agentic 不等于“让 LLM 自由搜索很多次”；必须有预算、状态、证据与可验证终止条件。

一次检索找答案，多次检索补证据链。

CHAPTER 9

第九章分布式搜索与工程系统

算法最终要跑在真实服务里：掌握分片、NRT、CDC、缓存、尾延迟和故障诊断。

题号	题目	难度
Q85	为什么 Search Index 要做 Sharding?	2/5
Q86	分布式 Search Query 的 Scatter-Gather 怎么工作?	3/5
Q87	为什么每个 Shard 只返回 Local TopK 可能有问题?	4/5
Q88	Primary Shard 与 Replica 的区别是什么?	2/5
Q89	Shard 越多是不是查询越快? 什么是 Over-sharding?	3/5
Q90	什么是 Near Real-Time (NRT) Search?	3/5
Q91	Refresh Interval 为什么存在 Freshness-Throughput Trade-off?	3/5
Q92	搜索索引如何与 MySQL/业务数据库保持同步?	4/5
Q93	搜索系统有哪些 Cache? 为什么 Query Result Cache 不一定最有效?	3/5
Q94	搜索延迟从 50ms 突然变成 2s, 怎么系统排查?	5/5

Q85 · 为什么 Search Index 要做 Sharding?

难度 2/5 频率 S 适用岗位分布式搜索

30 秒回答

单机无法承载超大索引的容量、QPS 与故障风险，因此按文档或路由键拆成多个 shard。一次 query 并发访问相关 shards，再聚合局部 TopK。

深度拆解

- 容量：索引大于单机 RAM/SSD。
- 吞吐：多 shard 并行查询，提高 aggregate QPS。
- 故障隔离：副本可接管。
- 但 shard 越多并非越好，fan-out 与协调开销会增加。
- 路由策略可按 hash 均衡，也可按 tenant/time/category 减少 fan-out，但会带数据倾斜风险。

- 按时间分 shard 的新闻搜索如何查询？
- 如何处理 hot shard？

Sharding 解决规模，但会把“单机问题”变成“分布式协调问题”。

分片不是免费并行。

Q86 · 分布式 Search Query 的 Scatter-Gather 怎么工作？

难度 3/5 频率 S 适用岗位 分布式搜索

30 秒回答

Coordinator 把 query scatter 到多个 shards，各 shard 本地检索 TopK，再 gather 回协调节点做全局 merge、可能的第二阶段 fetch/rerank。

深度拆解

- Query phase: 各 shard 返回 docIDs + scores 的局部 topK。
- Merge: coordinator 用 heap 合并成 global topK。
- Fetch phase: 只对最终文档取 stored fields/snippets，避免所有候选传大 payload。
- 尾延迟受最慢 shard/replica 影响，可用 hedged request 或 timeout/partial result。
- 请求 fan-out 大时 coordinator 本身可能成为瓶颈。

- 为什么 fetch 要分第二阶段？
- 一个 shard 超时怎么办？
- 如何减少跨机网络数据？

分布式搜索的 p99 常由 straggler 决定，而不是平均 shard latency。

Scatter 并行找，Gather 只合 TopK。

Q87 · 为什么每个 Shard 只返回 Local TopK 可能有问题？

难度 4/5 频率 A 适用岗位分布式排序

30 秒回答

全局 TopK 需要保证所有潜在赢家都被局部候选带回来。如果每 shard 的 K 太小、评分统计又不一致，可能截断真正的全局候选或产生 score comparability 问题。

深度拆解

- 简单同分布 additive scoring 下，每 shard 返回 K 通常足以合 global K，但实际有 rescore/fusion/feature fetch 等复杂阶段。
- 若后续 rerank 使用 shard 未参与的全局特征，local rank 与 global rank 可能差异很大，需要 over-fetch。
- BM25 的 shard-local DF/avgdl 可能造成 score 不可比，需要 global stats 或接受近似。
- Hybrid source fusion 也常需要从每路/每 shard 多取一些候选。
- 因此系统常设置 shard_size > final_size。

- 为什么 oversampling 能提升全局质量？
- Global IDF 如何获得？

不要机械说“local TopK 一定错误”；要说明什么条件下会出问题，以及为什么工程上常 over-fetch。

局部筛得太狠，会让全局排序没有选择。

Q88 • Primary Shard 与 Replica 的区别是什么？

难度 2/5 频率 A 适用岗位分布式搜索

30 秒回答

Primary 是某个 shard 的主写入副本，负责接收/协调写操作；Replica 保存冗余数据，提供容错并可分担读查询。

深度拆解

- Replica 提高 read capacity 与 availability，但也增加 storage 和 replication traffic。
- Primary 故障后可选 replica 提升。
- 复制通常是近实时而非严格零延迟，系统要定义 consistency semantics。
- 查询路由应做 replica load balancing，避免某台机器热点。

- 增加 replica 会提升 indexing throughput 吗？
- Replica lag 对 freshness 有何影响？

Replica 主要提升读与容错，不会免费提升写吞吐。

Primary 管写，Replica 管冗余和读扩展。

Q89 · Shard 越多是不是查询越快？什么是 Over-sharding？

难度 3/5 频率 S 适用岗位分布式性能

30 秒回答

不是。更多 shard 提高并行度，但每个 shard 都有 segment、heap、file handle、RPC 和 merge 成本；当 shard 太小，管理与 fan-out 开销超过并行收益。

深度拆解

- 单 query 要等更多 RPC，p99 更容易遇到 straggler。
- 每 shard 固定内存开销导致 heap/page-cache 被碎片化。
- 更多小 shard 通常意味着更多小 segments 与 metadata。
- 过度分片还增加 rebalance/recovery 操作数量。
- 合理 shard size 要结合索引大小、QPS、节点规格与恢复时间来压测。

- 什么时候应该 split shard？
- 什么时候应该 shrink/consolidate？

“多 shard = 多并行 = 更快”只在某个区间成立。

并行有收益，协调有成本。

Q90 · 什么是 Near Real-Time (NRT) Search?

难度 3/5 频率 S 适用岗位 Lucene / 系统

30 秒回答

文档被写入后通常先进入内存 `buffer/transaction log`，经过 `refresh/open new searcher` 后才对搜索可见，因此搜索可见性延迟是秒级/亚秒级的“近实时”，不是每次写后立即全局可见。

深度拆解

- Refresh 产生新的可搜索 `segment/view`，但不等同于持久化 `fsync/commit`。
- 频繁 refresh 提升 freshness，却增加小 `segment` 与 `merge` 压力。
- 不同系统可用 `refresh interval` 调节索引吞吐与可见性。
- 实时 `get` 与 `search visibility` 可能采用不同路径，因此语义要区分。

- Refresh 与 Flush/Commit 有什么区别？
- 为什么 refresh 不是免费的？

不要把 NRT 简化成“有 1 秒延迟”；具体延迟由 `refresh policy` 与负载决定。

NRT = 写入完成与搜索可见不是同一个时刻。

Q91 · Refresh Interval 为什么存在 Freshness-Throughput Trade-off?

难度 3/5 频率 S 适用岗位 Lucene / 系统

30 秒回答

更短 refresh interval 让新文档更快可搜索，但会产生更多小 segment、更多 searcher reopen 与 merge；更长 interval 提高批量写效率，却牺牲可见性。

深度拆解

- 写密集 bulk indexing 时常临时增大或关闭自动 refresh，再在批量结束后恢复。
- 新闻/库存场景需要低 freshness lag；历史文档库则可以更偏 throughput。
- 系统应监控 refresh time、segment count、merge time、indexing pressure。
- 如果业务允许，hot layer + cold layer 可以把 freshness 与大索引稳定性拆开。

- 为什么小 segment 多会拖慢 query？
- 实时商品库存是否一定要重建全文索引字段？

所有“更实时”的功能都有资源成本，必须明确业务 SLA。

Refresh 越快，新鲜越好；但索引越碎。

Q92 · 搜索索引如何与 MySQL/业务数据库保持同步？

难度 4/5 频率 S 适用岗位数据一致性

30 秒回答

不要依赖不可原子化的双写。常见稳健方案是数据库作为 source of truth，通过 CDC/binlog/Kafka 或 transactional outbox 把变更可靠地投递给 indexer，并保证幂等、版本和重放。

深度拆解

- CDC：捕获 DB commit 后的变更，天然与事务顺序对齐。
- Indexer 必须 idempotent：同一事件重复消费不能破坏状态。
- 用 version/sequence 防止乱序旧事件覆盖新文档。
- 定期 reconciliation/backfill 修复漏事件或历史 bug。
- Delete 事件同样要可靠传播，否则索引出现“幽灵文档”。
- 强一致通常成本很高；多数搜索接受 bounded eventual consistency。

- 为什么直接 DB+ES 双写会出错？
- Kafka 重放如何避免重复索引？
- Schema change 怎么处理？

一致性设计要从失败场景开始：DB 成功、Index 失败；Index 成功、DB 失败；消息重复/乱序。

数据库是真相，索引是可重建的派生视图。

Q93 · 搜索系统有哪些 Cache? 为什么 Query Result Cache 不一定最有效?

难度 3/5 频率 A 适用岗位性能

30 秒回答

缓存可以位于 query normalization、filter bitset、posting/block、feature、embedding、model result 与最终结果等多层。完整 query-result cache 对长尾 query 命中率可能很低，因此更细粒度复用常更稳。

深度拆解

- Head query 结果缓存收益巨大，但 freshness 与 personalization 会降低可复用性。
- Filter cache 适合高复用结构化条件，如 category/status。
- Embedding cache 对重复 query 或 session rewrite 很有价值。
- Feature cache 可缓存 document static features。
- OS page cache 其实是 Lucene 性能的重要组成，不应只盯应用层缓存。
- 缓存 key 必须包含 model/index/version、locale、permission 等影响结果的上下文。

- Personalized search 如何缓存?
- 缓存击穿如何处理?
- Index refresh 后哪些 cache 失效?

缓存设计的核心不是“加 Redis”，而是找到真正重复的计算与正确失效边界。

先测复用，再决定缓存层级。

Q94 · 搜索延迟从 50ms 突然变成 2s，怎么系统排查？

难度 5/5 频率 S 适用岗位性能诊断

30 秒回答

按请求链分层而不是先猜算法：入口/网络 → Query Understanding → shard fan-out → index/merge → feature store → model service → gather。先看 p50/p95/p99 与 stage tracing，再定位资源瓶颈。

深度拆解

- Query 分布是否变化：wildcard、超长 query、filter selectivity、ANN efSearch 是否被调大。
- Index：segment count、merge backlog、cache miss、shard imbalance、hot shard。
- Resource：CPU steal、GC、memory pressure、disk I/O、page fault、network retransmit。
- Model：GPU queue、batching、OOM/retry、feature RPC timeout。
- Distributed tail：少数 slow replica/shard 是否拖 p99；可看 per-shard waterfall。
- 发布变更：index version、model version、config、traffic shift 是否同时发生。

- 为什么平均延迟可能正常但用户觉得慢？
- 如何用 hedged request 降 tail？
- 什么指标能判断 merge 造成问题？

好的排障回答应是“证据驱动树”，而不是列十个可能原因。

先切 stage，再看 tail，再找资源或数据变化。

CHAPTER 10

第十章综合系统设计与 0→1 方法论

把前九章连接起来：高级搜索面试最终考的是约束下的系统权衡与持续优化能力。

题号	题目	难度
Q95	系统设计：从 0 设计一个 Google-like Web Search Engine	5/5
Q96	系统设计：淘宝 / Amazon 商品搜索	5/5
Q97	系统设计：亿级 Query Autocomplete	5/5
Q98	系统设计：10 亿文档 Semantic Search	5/5
Q99	系统设计：现代 Hybrid Search Engine	5/5
Q100	终极题：如果让你从 0 到 1 提升一个搜索引擎，你会怎么做？	5/5

Q95 · 系统设计：从 0 设计一个 Google-like Web Search Engine

难度 5/5 频率 S 适用岗位综合系统设计

30 秒回答

答题要同时覆盖 crawler/indexing、retrieval/ranking、distributed serving、freshness、quality/spam、logs/experiments。不要一上来画 Elasticsearch 集群，而应先做需求与数量级估算。

深度拆解

- 需求：文档规模、QPS、p99、freshness、语言、多租户/权限、相关性目标。
- 离线：crawl scheduler → parser → canonicalization/dedup → link graph/quality → index builder → shard publish。
- 索引：inverted index + vector index + document store + static feature store。
- 在线：query understanding → lexical/dense recalls → fusion → LTR → neural rerank → snippets。
- 分布式：sharding/replication、scatter-gather、cache、timeout/partial result、rolling index update。
- 数据闭环：judgments + click logs → debias → training → offline eval → A/B。
- 扩展问题：spam、safe search、canonical URL、fresh documents、duplicate results。

- 如何处理每天新增十亿页面？
- PageRank 在哪里算？
- 如何做跨语言搜索？
- 如果 p99 要 100ms，Cross-Encoder 怎么放？

系统设计最忌“组件堆砌”。每个组件都要回答：解决什么瓶颈、输入输出是什么、失败如何降级。

先定约束，再定索引，再定 **cascade**，最后闭环。

Q96 · 系统设计：淘宝 / Amazon 商品搜索

难度 5/5 频率 S 适用岗位电商搜索

30 秒回答

电商搜索的关键不是只做文本相关性，而是把 query 解析成类目/品牌/属性/价格等结构化意图，并联合库存、销量、质量、CTR/CVR 等业务信号排序。

深度拆解

- Query Parsing: ‘3000 以内黑色轻薄游戏本’ → category=laptop, price≤3000, color=black, attributes=thin,gaming。
- Index: title/description 倒排 + brand/category/attribute filters + dense embeddings + inventory/price doc values。
- Recall: lexical、attribute exact、dense、personalized、hot/new item。
- Rank: relevance 是基础，叠加 CTR/CVR、quality、price competitiveness、delivery、seller reliability。
- 约束: out-of-stock 过滤、类目多样性、广告标识、重复 SKU 合并。
- 在线实时性: price/inventory 变化频繁，尽量通过实时 feature/doc-values 层更新而不是全量重建全文字段。

- 如何处理商品标题关键词堆砌？
- 个性化权重过大有什么风险？
- 新商品没有 CTR 怎么排？

电商搜索不能让转化目标压过 relevance，否则会变成“推荐广告列表”。

先理解商品属性，再做文本与业务的联合排序。

Q97 · 系统设计：亿级 Query Autocomplete

难度 5/5 频率 A 适用岗位 Query Suggest

30 秒回答

设计要覆盖候选索引、热度流式更新、个性化、安全与极低延迟。最稳的框架是离线全局 FST + 实时 hot layer + 用户历史层，最后轻量 ranker 融合。

深度拆解

- 离线从 query logs 聚合去隐私化的频率/质量，构建 prefix → topN completion。
- 全局索引用 FST/压缩 trie；实时趋势用内存 KV/streaming topK。
- 用户历史候选本地/用户缓存读取，避免把私人 query 放入全局库。
- Rank features: prefix length、frequency、recency、trend、locale、personalization、edit distance。
- 系统目标常是 p99 20-50ms 级，因此不能依赖重型在线模型。
- 防刷与安全：低频 query、个人信息、违规词、机器人流量需要过滤。

- 如何做 typo tolerant autocomplete?
- 热点事件如何秒级生效?
- 如何估算 FST 内存?

Autocomplete 是“每个 keystroke 的搜索”，它的 QPS 和 latency 压力往往比主搜索更高。

全局静态 + 实时热点 + 个人历史，三层候选融合。

Q98 · 系统设计：10 亿文档 Semantic Search

难度 5/5 频率 S 适用岗位 Vector Search

30 秒回答

先做资源预算，再选索引。1B×768 float32 仅原始向量约 3.07TB，因此通常需要分片、压缩或降维；再根据 recall/p99/QPS 选择 HNSW、IVF-PQ 或混合架构。

深度拆解

- 需求：metric、TopK、target Recall@K、QPS、p99、update rate、memory budget、GPU/CPU。
- Embedding pipeline：versioned model → batch encode → index build → shadow validation → atomic publish。
- Index：若内存充足且重 recall，可 HNSW；极大规模/强压缩偏 IVFPQ/OPQ；热点可用 HNSW、冷数据用 IVF-PQ。
- Shard：hash/tenant/semantic routing；查询并发 fan-out，局部 ANN topK 后全局 merge。
- 更新：新向量增量层 + 周期性主索引 rebuild，避免分布漂移。
- 评估：ANN recall（相对 exact ground truth）与 retrieval relevance 分开测。

- 128GB/节点、32 节点够不够？
- PQ code 32 bytes 时 1B 需要多少空间？
- 如何无停机切 embedding model？

语义搜索设计不能只说“用 Milvus/FAISS”；真正考的是数量级与 trade-off。

先算 3TB，再谈索引。

参考：R6

Q99 · 系统设计：现代 Hybrid Search Engine

难度 5/5 频率 S 适用岗位 Hybrid / RAG

30 秒回答

理想答案要把 lexical、dense、fusion、reranker、LTR、权限过滤和可观测性统一起来，并说明什么时候走哪条通道。

深度拆解

- Query Understanding 输出多个 lexical/dense rewrites 与结构化 filters。
- BM25 与 ANN 并行召回；必要时 learned sparse 作为第三路。
- 候选 merge/dedup 后用 RRF 或 learned fusion；保留 source/rank 特征。
- PreRank 过滤到几百条，Cross-Encoder/ColBERT 做语义精排；最终 LTR/constraint rerank。
- 权限过滤最好尽量早做，但 vector ANN 的 filtered search 需要专门设计，避免 post-filter 造成 recall 空洞。
- 在线监控：每路 recall contribution、fusion overlap、NDCG、latency、zero-result、ANN recall、GPU queue。

- Filter-first 与 ANN-first 有什么 trade-off?
- RRF 后还要不要 LTR?
- 如何动态决定是否启用 dense 通道?

Hybrid 的难点不是“两路都查”，而是 query routing、filtering、fusion、rerank 和成本控制。

Hybrid Search = 多路召回 + 可解释融合 + 强重排。

Q100 · 终极题：如果让你从 0 到 1 提升一个搜索引擎，你会怎么做？

难度 5/5 频率 S 适用岗位综合 / Staff

30 秒回答

最高分答案不是“换更大的模型”，而是建立诊断闭环：先定义成功指标和评测集，建立 error taxonomy；再从 query understanding、recall、ranking、bias、latency 逐层定位瓶颈；每次改动用离线与 A/B 验证，并沉淀日志与训练飞轮。

深度拆解

- 第 1 步：建立 golden query set + human judgments + production slices；统一 Recall@K、NDCG@K、MRR、latency 等指标。
- 第 2 步：做错误分类：zero result、intent error、spelling、missing recall、ranking inversion、freshness、duplicate、policy。
- 第 3 步：做 oracle analysis，判断当前上限卡在 recall 还是 rank；不要盲目换模型。
- 第 4 步：从最便宜高 ROI 的改动开始：analyzer/rewrite/synonym、BM25 field、new recall channel、LTR features。
- 第 5 步：引入 Dense/Hybrid/Cross-Encoder 时建立 cost-quality frontier 和 fallback。
- 第 6 步：点击日志做去偏，模型训练做 point-in-time features，避免 leakage。
- 第 7 步：A/B + guardrails；失败实验也沉淀 slice 与 error case。
- 第 8 步：Search Flywheel: traffic → logs → mining → labels → training → evaluation → deployment → traffic。

- 如果只能做一周，你优先做什么？
- 如何判断是数据问题还是模型问题？
- 离线指标完全没有人工标注怎么办？
- 如何向业务解释“模型更复杂但不值得上”？

Staff 级面试真正考的是优先级和闭环：能否把算法、数据、系统、实验变成持续改进机制。

先测量，再定位；先修瓶颈，再上复杂模型。

附录 A：核心公式速查

面试公式不是为了背诵，而是为了能在白板上快速恢复“变量变化会让结果怎么变”。下面把最常用的 7 组公式压缩到一页，建议每个公式至少能讲出两个极限情况。

BM25

$$\text{BM25}(D, Q) = \sum_{q_i \in Q} \text{IDF}(q_i) \frac{f(q_i, D)(k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + b \frac{|D|}{\text{avgdl}}\right)}.$$

记忆： k_1 控制 TF 饱和； b 控制长度归一化。

NDCG

$$\text{DCG@K} = \sum_{i=1}^K \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}, \quad \text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}.$$

RankNet / LambdaRank

$$P_{ij} = \sigma(s_i - s_j).$$

LambdaRank 在 pairwise gradient 上乘以与 $|\Delta \text{NDCG}|$ 相关的权重。

Dense Contrastive Loss

$$L = -\log \frac{e^{s(q, d^+)/\tau}}{e^{s(q, d^+)/\tau} + \sum_j e^{s(q, d_j^-)/\tau}}.$$

RRF
$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + r(d)}.$$

Unit Vector 下的 Cosine / IP / L2
$$\|x - y\|^2 = 2 - 2x^T y.$$

IPS
$$\hat{R} = \frac{1}{n} \sum_i \frac{c_i}{p_i} \ell_i.$$

附录 B：系统设计答题模板

1. 需求：文档规模、QPS、TopK、p99、freshness、update rate、权限、安全、业务目标。
2. 估算：索引大小、向量大小、网络 fan-out、模型推理次数、存储副本。
3. 数据面：ingestion、清洗、schema、index build、version、publish、backfill。
4. 查询面：query understanding、recall、fusion、rank、rerank、fetch。
5. 分布式：sharding、replica、routing、timeout、partial result、hot shard。
6. 一致性：CDC、幂等、version、delete、reconciliation。
7. 评估：Recall/NDCG + p95/p99 + cost + A/B guardrails。
8. 降级：model timeout、vector service down、feature miss、shard slow 的 fallback。

CHAPTER 13

附录 C：14 天冲刺计划

天数	主题	任务
Day 1	全局架构	Q1-10; 画 3 次完整 pipeline
Day 2	倒排基础	Q11-16; 手写 posting intersection
Day 3	Lucene	Q17-20; 解释 segment/merge/NRT
Day 4	BM25	Q21-30; 白板推公式与极限情况
Day 5	Query	Q31-40; 设计中文电商 query parser
Day 6	LTR-I	Q41-45; Point/Pair/List + RankNet
Day 7	LTR-II	Q46-52; LambdaRank/LambdaMART + leakage
Day 8	Metrics	Q53-62; 手算 NDCG + Click Bias
Day 9	Dense	Q63-68; InfoNCE + hard negatives
Day 10	ANN	Q69-74; 做 1B 向量内存估算
Day 11	Hybrid	Q75-81; RRF/SPLADE/Cross/ColBERT
Day 12	RAG/Infra	Q82-94; 端到端故障树
Day 13	系统设计	Q95-99; 每题 30 分钟白板
Day 14	模拟面试	Q100 + 随机抽 20 题, 只看题目脱稿答

CHAPTER 14

附录 D：参考资料

这些资料用于核验本书关键公式、算法定义与当前实现参数。书中解释均为原创归纳，引用资料只作为技术依据。链接均可直接点击。

编号	资料	链接
R1	Christopher D. Manning, Prabhakar Raghavan, Hinrich Schütze. Introduction to Information Retrieval. Cambridge University Press, 2008.	打开
R2	Apache Lucene. BM25Similarity (Lucene 10.x API): k1, b, IDF implementation and defaults.	打开
R3	Chris J. C. Burges. From RankNet to LambdaRank to LambdaMART: An Overview. Microsoft Research Technical Report, 2010.	打开
R4	Thorsten Joachims, Adith Swaminathan, Tobias Schnabel. Unbiased Learning-to-Rank with Biased Feedback. WSDM 2017.	打开
R5	Yu. A. Malkov, D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. 2016/2018.	打开
R6	Meta / Facebook Research. Faiss Wiki: Index types, IVF, PQ, HNSW, index selection and memory trade-offs.	打开
R7	Omar Khattab, Matei Zaharia. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. SIGIR 2020.	打开
R8	Thibault Formal et al. SPLADE v2: Sparse Lexical and Expansion Model for Information Retrieval. 2021.	打开
R9	Gordon V. Cormack, Charles L. A. Clarke, Stefan Büttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009.	打开
R10	Rodrigo Nogueira, Kyunghyun Cho. Passage Re-ranking with BERT. 2019.	打开
R11	Vladimir Karpukhin et al. Dense Passage Retrieval for Open-Domain Question Answering. EMNLP 2020.	打开
R12	Nils Reimers, Iryna Gurevych. Sentence-BERT. EMNLP 2019.	打开
R13	Nandan Thakur et al. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. 2021.	打开
R14	Elastic. How full-text search works: analysis, inverted index and BM25-based scoring.	打开
R15	Stephen Robertson et al. Okapi at TREC-3. TREC 1994.	打开

结语：把 100 题压缩成 10 个问题

如果只能带 10 个问题进面试现场，请记住：

1. 用户到底想找什么？
2. 相关文档有没有进入候选？
3. 为什么这个候选分数高？
4. 哪个 ranking error 对用户最贵？
5. 日志中的点击到底有多偏？
6. 新模型的收益是相关性收益，还是候选数量/数据偏差造成的假象？
7. 召回、排序、延迟、成本的 Pareto frontier 在哪里？
8. 索引更新和模型更新如何无停机、可回滚？
9. 离线提升能否通过 A/B 兑现？
10. 下一次失败实验能否让系统更容易诊断？

搜索算法岗的核心能力，不是“知道很多模型”，而是能在海量候选、有限延迟、有偏日志和不断变化的数据上，持续把正确结果稳定地放到用户最先看到的位置。